

An R companion

Eduardo Montoya

2026-10-04

Contents

Preface	5
1 Introduction: Getting Familiar with R and RStudio	7
1.1 What is R and RStudio?	7
1.2 Downloading and installing R	8
1.3 Downloading and installing RStudio	8
1.4 The layout of RStudio	9
1.5 Expressions and Assignments	11
1.6 Entering and running R code in RStudio	12
1.7 A fancy calculator	13
1.8 Working with vectors	14
1.9 Data types in R	16
1.10 Comments in R	17
1.11 Using R functions	18
1.12 Loading a package and installing new packages	19
2 Some basic R tasks	21
2.1 Using R's built-in datasets and "viewing" datasets	21
2.2 .csv files	22
2.3 Importing a .csv file	24
2.4 Using RStudio's data import feature	26
2.5 Summarizing data	26
2.6 Export an R plot or R output	28

2.7	Extracting variables from dataframes	30
2.8	Understanding common R errors	31
2.9	Help files	33
3	R code for Regression Analysis	37
3.1	Linear Regression with One Predictor Variable	37
3.2	Inference in simple linear regression	47
3.3	Residual analysis and remedial measures	54
3.4	Least absolute deviation (LAD) regression	65
3.5	The matrix approach to simple linear regression	65
3.6	Multiple linear regression (MLR)	72
3.7	Interactions	80
3.8	Standardized multiple regression	81
3.9	Polynomial regression	83
3.10	Model selection	85
3.11	More diagnostics	99

Preface

This document is freely available. There are grammatical errors in the document. If you do encounter a mistake or an error please inform me so that it can be corrected. If you would like to add material or have suggestions for material changes, then please contact me to include them in any future updates, and any contributions will be acknowledged. Keep in mind that this document is a work in progress, and the eventual goal is to make this document independent of any class textbook.

The first two chapters introduce one to the **R** environment and **RStudio** and some basic **R** functions so that one gets an idea of the ““Look and Feel” of **R** and **RStudio**. The last chapter discuss how to use **R** to carry out the analyses and techniques for some common areas covered in undergraduate regression analysis.

The best way to use this document is to apply a learn-by-doing approach. That is, don't just read through the **R** code given, but try the sample code given so that you understand what works and what does not. Finally, this document is not intended to be a know all guide but only a companion to your course. It is also worth noting that there are several ways to do the same thing in **R** and my hope is that I presented the easiest way(s).

Chapter 1

Introduction: Getting Familiar with R and RStudio

1.1 What is R and RStudio?

R is a GNU project and may be thought of as an implementation of the **S** language (developed at Bell Laboratories by Rick Becker, John Chambers and Allan Wilks). It is a language and environment for statistical computing and graphics. **R** contains a large number of built-in functions for classical and modern statistical analysis. One of the main advantages of using this program, particularly for undergraduate students, is that it is free! A core team of statisticians and countless other contributors consistently maintain, update, and improve **R** and make versions that run well on most operating systems. The web page for the **R** Project for statistical computing is located at <https://www.r-project.org/>.

In this course, we will run **R** through **RStudio**. **RStudio** is an integrated development environment (IDE) that allows you to interact with **R** more readily. **RStudio** is also free to use. For our purpose, we will be using **RStudio** as a graphical user interface for **R**. Think of **R** as the engine for a car, whereas **RStudio** is the dashboard, wheel, etc. that allows one to control the engine (although it does so much more such as create slides, books, web applications, and other things).

1.2 Downloading and installing R

The CSUB virtual computer lab will have R and RStudio installed. If you prefer to install these on your personal computer, follow these steps:

1. Visit the site <https://cran.r-project.org/>.
2. Click the link that corresponds to your appropriate operating system. (see Figure 1.1). The directions given are for Windows but the process is fairly similar for Mac or Linux. Chromebook users will have to use the virtual computer lab to access R and RStudio unless you are proficient in using Linux.
3. Select *base* and download the latest release (the link will state “Download R ‘some version’ for Windows”)
4. Once the program has downloaded, install **R** with the default settings.
5. That’s it!

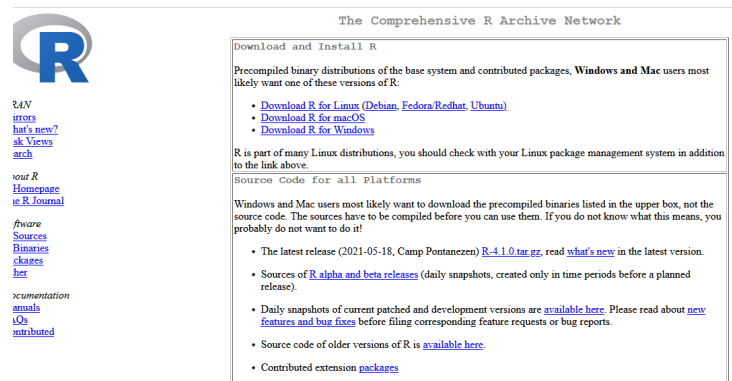


Figure 1.1: Figure 1.1: R download page

1.3 Downloading and installing RStudio

Follow the steps:

1. Visit the RStudio IDE downloads page.
2. Download the RStudio Desktop installer for your operating system. The screenshots below show an earlier version of the download pages; the current layout may differ.

3. Once the program has downloaded, install **RStudio** with the default settings.
4. That's it! You can now run **RStudio** from your home computer.

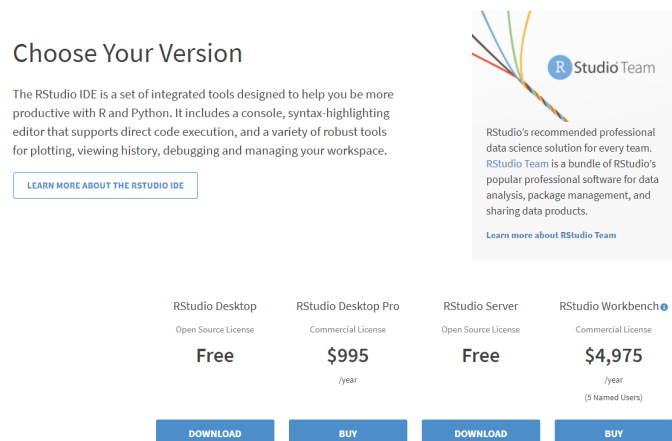


Figure 1.2: Figure 1.2: RStudio download page

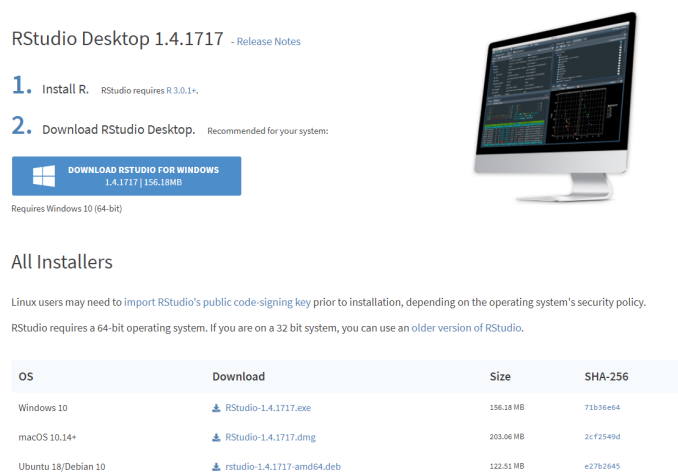


Figure 1.3: Figure 1.3: RStudio Desktop download page

1.4 The layout of RStudio

Open **RStudio**. The first time you open **RStudio**, you will see three panes (Figure 1.4). A fourth pane is hidden by default, but can be opened by clicking

the File drop-down menu, then New File, and then R Script (*File>New File>R Script*). **RStudio** should look something like what you see below but perhaps with a different color scheme. Please see me during student hours or before/after lecture if you wish to change the color scheme.

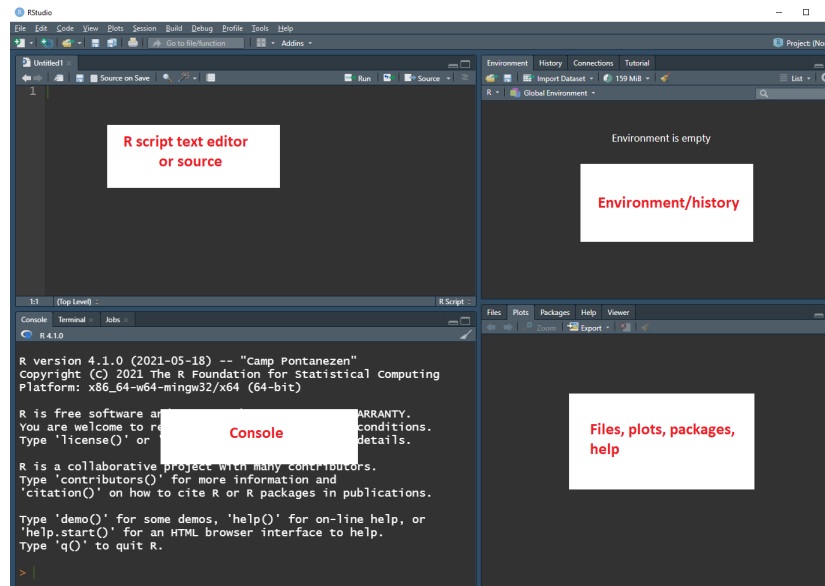


Figure 1.4: Figure 1.4: RStudio layout.

A brief description of each pane:

- The upper left pane: The **R Script text editor** or **source**. This is where commands are written before being sent to the console for execution. Writing and saving a series of **R** commands in the script file makes it easy to reuse or modify code at a later time.
- Lower left pane: the **console** pane. Every time you launch **RStudio**, it will have the same text at the top of the console telling you the version of **R** that you're running. Below that information is the **R prompt** (the symbol `>`). As its name suggests, this prompt is really a request, a request for a command. It is here where **RStudio** will tell **R** what to do. This is the most important pane because this is where **R** actually does stuff and provides output. That is, it is where commands are entered and executed with output printed.
- Upper right pane contains your **environment/history** pane.
 - In the *environment* tab you can see which data and values **R** has in its memory. **R**'s memory is called an **environment**.

- The *history* tab shows what has been typed before.
- Don't worry about the rest of the tabs.
- Bottom right pane is the **files/plots/packages/help** pane.
 - In the *Files* tab you can browse and select files to open.
 - The *Plots* tab will show any plots that you create.
 - The *Packages* tab shows a list of installed R packages (more on R packages later).
 - The *Help* tab is the output location of any help files called (more on help files later).

1.5 Expressions and Assignments

R commands can evaluate expressions or assign their results to objects. At the command prompt `>` in the console pane, do the following:

- Type `25-5` and hit enter. (this is an example of an expression).
- Type `h=25-5` and hit enter (this is an example of an assignment). This can be read as the difference between 25 and 5 is assigned to the object `h`. Next type `h` and hit enter.
- Type `H<-20` and hit enter. Next type `H` and hit enter.

The result should be as follows

```
25 - 5
#> [1] 20
h = 25 - 5
h
#> [1] 20
H <- 20
H # The pound sign is used for comments
#> [1] 20
```

Notice the following from the commands that were executed above:

- `h` and `H` are not the same thing (they are clearly distinct), so **R** is case sensitive.
- You can also see these new objects (`H` and `h`) are in your environment tab on the upper right pane. These objects are generally called *R objects*.
- You will get into the habit of saving things in *R objects* so that we can access them at a later time. During an R session, objects remain in memory unless you remove them.

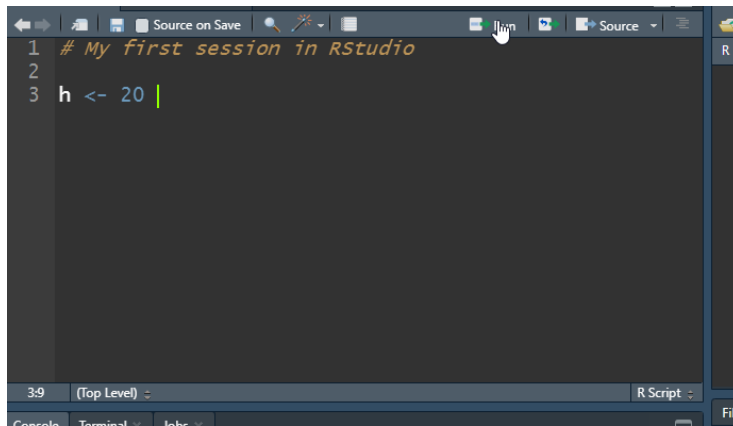
- When a session ends, objects are not retained unless you save them. **RStudio** may restore a previously saved workspace at startup, depending on its settings. Saving and rerunning your R script lets you recreate the objects.
- The assignment operator is `=` or `<-`. Either one is acceptable, although I have a preference for `<-` so most of my handout/notes will reflect this.
- Ordinary R object names start with a letter or a period that is not followed by a digit. The remaining characters can include letters, digits, periods, and underscores. Reserved words such as `if` cannot be used as ordinary object names.
- Outside a quoted string, `#` begins a comment. R ignores the rest of that line.

Back to the *R objects* `H` and `h`. The objects were created and are now stored in **R**'s memory. So if you type `H` and hit enter again, then 20 will come up. **RStudio** can be closed by selecting *File > Quit Session* or you can just enter the command `q()` in the console. When exiting **RStudio** you will be asked "Save workspace image to ...". If you select "Don't Save", then you will lose everything that is in your memory. For example, if you were to close without saving and then open **RStudio** again and type `H` you will get an error such as **Error: object 'H' not found**, unless a saved workspace restores it. Saving the workspace is not required for these examples.

1.6 Entering and running R code in RStudio

The easiest way to enter code or commands into **RStudio** is to type a command in the console pane and press enter. But this method is only efficient for short and simple analyses. The recommended way is to use **RStudio**'s text editor. Use the built in text editor as follows:

- Open an R script file (*File > New file > R script*).
- Type in your **R** code or commands into this editor. For example, type `h <- 20`.
- Save the R script file as an *.R* file. For example, save your file as *FirstR-session.R*.
- You may then execute the command `h <- 20` directly from the text editor without copying and pasting the command. This is done by highlighting the command, and then selecting the "Run" button (the figure below). You should then see the command executed in the console pane.



Important: Always save your R script file periodically so that you may re-run the analysis when necessary (this applies to any assignments or exams).

1.7 A fancy calculator

At its most basic level, **R** can be viewed as a fancy calculator. While we won't be using **RStudio** to do algebra problems, it is a good way to get familiar with **RStudio**. The basic operations are `+` (add), `-` (subtract), `*` (multiply), `/` (divide), and powers with the `^` operator. R follows the order of operations. Type the following on the new line in the editor and select "Run":

```
727/163
```

and you should get the output `[1] 4.4601227`.

Upon selecting "Run", the result of the above appears in the console pane, preceded by the command you executed, and prefixed by the number 1 in square brackets `[1]`. The `[1]` indicates that this is the first (and in this case only) result from the command. Many commands will return multiple values. Try the following one by one, where each is typed on a new line and then selecting "Run" after typing the command in the editor:

```
25*10
```

```
5/2
```

```
2 + 2
```

After running these commands, you should note the following about the R prompt, `>`:

- The `>` prompt means that **R** is content and ready for a new command or input.
- Please note that spacing is not an issue with **R**. For example, `5 / 2` is the same as `5/2`. However, using spaces makes the code **easier** to read and catch mistakes.

Don't worry about saving your workspace as we generally will not need to do this in the course.

1.8 Working with vectors

The `c( )` function combines or concatenates terms together into a vector. Suppose we wish to store the values 1, 0, 2, 0, and 3 in a vector and store this result in an object called `x`. To do so, run the following:

```
x <- c( 1 , 0 , 2 , 0 , 3 )
x
```

```
## [1] 1 0 2 0 3
```

What is the mean of the data stored in `x`? We can do this by summing up the values and dividing by 5:

```
SumOfx <- 1 + 0 + 2 + 0 + 3
SumOfx/5
```

```
## [1] 1.2
```

We could also use the `sum( )` function which will add all the elements in a numerical vector:

```
sum(x)
```

```
## [1] 6
```

```
sum(x)/5
```

```
## [1] 1.2
```

Note the above in summation notation is

$$\frac{\sum_{i=1}^5 x_i}{5}$$

Suppose we wanted to compute the sum below:

$$\sum_{i=1}^5 x_i^2$$

We can do this in steps before we use the `sum( )` function:

```
x^2

## [1] 1 0 4 0 9

xsq <- x^2 # assign the squared elements to 'xsq'
sum( xsq )

## [1] 14
```

What if we wanted to compute the sum below?

$$\frac{\sum_{i=1}^5 (x_i - \bar{x})^2}{4}$$

Like before, we do this in steps:

```
diffofxANDxbar <- x - sum(x)/5
diffofxANDxbar

## [1] -0.2 -1.2 0.8 -1.2 1.8

sqdiffofxANDxbar <- diffofxANDxbar^2
sqdiffofxANDxbar

## [1] 0.04 1.44 0.64 1.44 3.24

sum( sqdiffofxANDxbar ) / 4

## [1] 1.7
```

1.9 Data types in R

R has six atomic vector types: double (usually called numeric), integer, complex, character, logical, and raw. A factor is a class for categorical data, stored as integer codes with labelled levels.

We will only deal with numeric, character, factor and logical data types. Numeric data consists of decimal values. For example,

```
j <- 10.355
j
```

```
## [1] 10.355
```

In **R**, numeric is the default type for numbers. An ordinary numeric literal is stored as a double; integer values can be created explicitly, for example with `10L`. Character data are strings, which can contain zero or more characters. Character data are created by putting the string in quotations. For example,

```
k <- c( "hi", "hello" )
k
```

```
## [1] "hi"    "hello"
```

```
L <- "3.4403" # a string since we put it in quotations!
L # this is not numeric
```

```
## [1] "3.4403"
```

Logical values are **TRUE** or **FALSE**; missing logical values are represented by **NA**. Logical values are generally created when there is a comparison between variables.

```
p <- c( TRUE, TRUE, FALSE ) # no quotation marks!
p # this is not character data but rather logical values
```

```
## [1] TRUE TRUE FALSE
```

Factor data contains a set of numeric codes with character-valued levels. A simple way to create a factor variable is to first define it as a character variable and then convert it to a factor variable by applying `as.factor( )` to the vector:

```
MaritalStatus <- c( "married", "married", "divorced", "single", "single", "widowed", "married" )
MaritalStatus
```

```
## [1] "married" "married" "divorced" "single" "single" "widowed" "married"
```

```
### convert character variable to factor variable:
```

```
MaritalStatus <- as.factor( MaritalStatus )
MaritalStatus
```

```
## [1] married married divorced single single widowed married
## Levels: divorced married single widowed
```

An atomic vector has one underlying type. If you combine values of different types with `c()`, R generally converts them to a common type. For example, `c(1, "two")` produces the character vector `c("1", "two")`. Further, you are not expected to be able to create factor variables (or other data types), but rather you are expected to only recognize and distinguish between these data types.

1.10 Comments in R

Outside a quoted string, everything after `#` on a line is a comment and is ignored by **R**. Adding comments to your R script is useful because it will help you recall what your commands or lines of code will do. In the editor, type the following and select “Run”:

```
### Comments are ignored by R!
```

```
y = 1 + 3 # this command tells R to compute 1 plus 3 and assign it to an R object called y
```

Note that `y` is now in **RStudio**’s memory, and we can access its value by typing `y` on a new line and selecting “Run”. Now, type the following in your editor on a new line and then select “Run”:

```
# z = 3 - 7 # this code is ignored because of # `
```

Note that this command is ignored because of `#`. Get into the habit of using comments!

1.11 Using R functions

R has many built-in functions. For example, the `c( )` function combines, or concatenates terms together into a vector.

Suppose we wish to assign the values

```
1 0 2 0 3 1 0 1 2 0
```

to a vector denoted by **X**. In your R script, run the code given below:

```
X <- c(1, 0, 2, 0, 3, 1, 0, 1, 2, 0)
```

```
X
```

You should have obtained output as follows:

```
X <- c(1, 0, 2, 0, 3, 1, 0, 1, 2, 0)
```

```
X
```

```
## [1] 1 0 2 0 3 1 0 1 2 0
```

The output appears below the code. The `#>` prefix in these notes distinguishes output from code; it is not part of the value printed in the R console. To recall earlier console commands, use the up and down arrow keys.

Another example of an **R** built-in function is the `sum( )`. This function computes the sum of all the numbers in a vector. The syntax consists of the function name followed by parentheses to contain the argument(s). For this example, pass the data vector to `sum()`. The function can also sum several arguments. We will make use of R's many built in functions along the way. Examples of other functions are shown below:

```
### computing the mean of the numbers stored in X
```

```
mean(X) # sample mean
```

```
#> [1] 1
```

```
### compute the length of the vector
```

```
length(X)
```

```
#> [1] 10
```

```
# more on functions later.
```

1.12 Loading a package and installing new packages

R's built-in functions are supplied by packages; you can also define your own functions. There are many *packages* that can be installed to expand the ability of R. By default, R comes with several packages installed (some loaded automatically when R starts and some not). For example, a function called `mean()` is available from the **base** package, and this package is loaded automatically when R starts, so we can use this function without loading an additional package.

As for those packages that aren't loaded automatically, we first have to load a package to use its "tools". For example, the **MASS** package comes with **R** but is not automatically loaded. This package provides a function called `boxcox()` (for Box-Cox transformations). To use `boxcox()`, we first have to load this package by running the command `require(MASS)` or `library(MASS)`.

Remember to think of *packages* as toolboxes. We will install additional *packages* called **openintro** and **mosaic** that will provide useful data sets and functions ("tools"). These packages are specifically designed to make **R** more accessible.

To install these packages, in the **R** console type the following command:

```
install.packages( "openintro" ) # no spaces within quotation marks!
```

Hit enter, and you will see activity occurring in the **R console**. This may take a few minutes to complete. Note the following during the installation process:

- If R asks you to choose a CRAN mirror for downloading, select any *USA* mirror.
- If R asks to use your personal library, answer yes.
- If R prompts you to update any packages, please agree to the updates.

When it is done installing, check that the console reports no installation error. A message about downloaded packages alone does not confirm that every package installed successfully.

Now install the **mosaic** package. In the **R** console type and run the following command:

```
install.packages( "mosaic" ) # no spaces within quotation marks!
```

Again, this may take a few minutes to complete. Note that these packages only have to be installed once on your computer. However, every time a new RStudio session is started, the package will have to be loaded before datasets and/or functions from the package can be used. If a different computer is being used, these packages will have to be installed again. For now, only these additional packages are required.

Chapter 2

Some basic R tasks

By now, the **RStudio** environment should look familiar. This chapter covers basic R tasks: loading and importing data, summarizing data, using help files, and understanding common errors. Recall that an assignment such as `H <- 20` creates an R object named `H` and stores the value 20 in it.

2.1 Using R’s built-in datasets and “viewing” datasets

Take a look at the Environment tab in the top right pane. Datasets that you import or load into the global environment appear in this tab. Some built-in datasets can also be used directly without first appearing there. In this course, we will either import a dataset that is stored in a .csv file or use a dataset that already comes with **R** (by way of an **R** package). Here, we illustrate using a dataset that already comes with R. The dataset is the resulting measurements from an experiment on the effect of diet on early growth of chicks.

R refers to this dataset as `ChickWeight`. We can look at this data by first telling **R** that we want to use it by using the following command:

```
data( ChickWeight ) # this tells R that we want to use the 'ChickWeight' data
# It should now appear in R's memory (see Environment tab)
# data() loads the dataset; View() opens the data viewer.
```

We can now look at the data within the console by typing its name in the R script, highlighting it, and selecting “Run”

```
ChickWeight
```

It's going to print a lot of data (578 rows of it). To view the data in a spreadsheet, click on the name `ChickWeight` in the Environment tab (upper right pane). This will open a spreadsheet of the data in a new tab in the source pane.

The data viewer shows four variables: `weight` (body weight in grams), `Time` (days since birth), `Chick` (the chick identifier), and `Diet` (diet group 1, 2, 3, or 4). Each of the 578 rows represents a measurement; the same chick is measured on multiple days. Row numbers are displayed separately and are not a fifth variable. Use the scroll bar in the viewer to examine the complete dataset.

Alternatively, to open the data spreadsheet, one can instead run:

```
View( ChickWeight )
```

Note that the row numbers in the first column are not part of `ChickWeight`'s data. R adds them as part of its printout to help you make visual comparisons. You can think of them as the index that you see on the left side of an Excel spreadsheet. In fact, the comparison to a spreadsheet will generally be helpful. R has stored `ChickWeight` dataset in a kind of spreadsheet or table called a **dataframe**. Rather than looking at the spreadsheet of the data, we can instead explore the **dataframe** in various ways to extract useful characteristics. For example, one can see how many variables (chick characteristics of interest) were measured/recorded and the number of observations on these variables by running:

```
require( mosaic ) # tell R we want to use the mosaic package  
pillar::glimpse( ChickWeight )
```

The `pillar::glimpse()` function is provided by `pillar` and re-exported by `dplyr`. Use `pillar::glimpse()` to call it explicitly.

In general, datasets that come with R or an R package, such as `ChickWeight` have what is called a *help file* that will provide details regarding the data set. Help files are discussed later in this chapter.

2.2 .csv files

It is assumed you have used an Excel spreadsheet. If not, please view the tutorial Excel Tutorial for Beginners .

Data sets used in this class will either be built-in datasets (that is, they come with R or from an R package) or they will be *.csv* files. A *.csv* may be viewed

with Microsoft Excel. We will import .csv files into RStudio. If you have data that you want to import into RStudio, you will first store your data in a .csv file using Excel. The format in which you store it should be a **data matrix** or in **tidy form**. There are three characteristics that make a dataset a data matrix or tidy:

- Each variable must have its own column.
- Each observation must have its own row.
- Each value must have its own cell.

To set up your spreadsheet, do the following steps:

- Step 1 – Input and organize your data in Excel.
 - Organize your data in an Excel worksheet, such that the first row (Row 1) contains the column names and each subsequent row contains all the necessary information for each data point. For example:

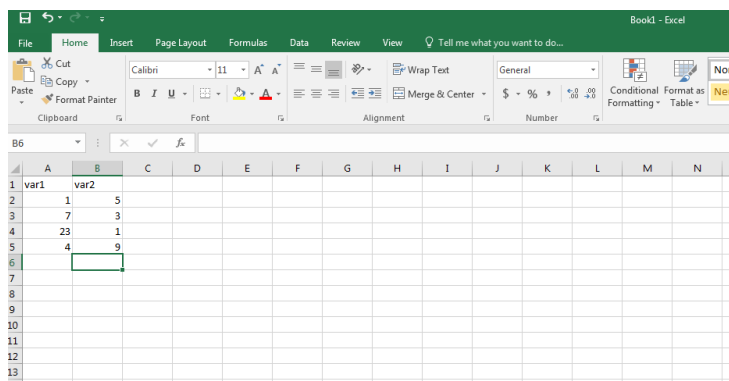


Figure 2.1: Figure 2.1

- Step 2 – Save your worksheet as a comma separated values (.csv) file type. Select the following Excel tabs *File>Save As*, then select where you want to save your .csv file. A window will appear in which one can specify their desired *filename* as well as their desired *file type*. Enter a file name and click on “Save as type” and scroll down to select **CSV (Comma delimited)** (see figure below).

When you click to save the worksheet as a comma separated file (.csv), Excel may present you with one or two warnings. One may warn you that the .csv format cannot accommodate multiple worksheets. That’s fine; just click “Yes” to save the active worksheet. The second warning tells you that “Some features

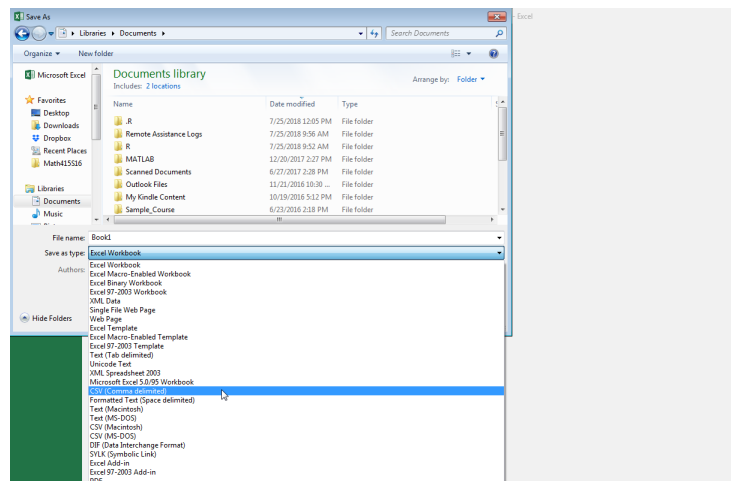


Figure 2.2: Figure 2.2

in your workbook might be lost if you save it as a CSV (Comma delimited).” Again, that’s fine, just click “Yes”. When you’re done, you will see that a new file has been created in the directory where you saved your file. If you were to open this file in Excel you would see your spreadsheet with the data you entered. On the surface, it looks no different than an Excel spreadsheet file (.xls or .xlsx)

Important: The examples in this chapter assume that the first row of the CSV contains column names and each subsequent row contains one observation. R can also read files without column names by using `header = FALSE`.

2.3 Importing a .csv file

We will use the **R** function `read.csv()` to import data into **RStudio**. We illustrate the method using the “MLB salaries (2010)” data set and this data is stored in a .csv file. This .csv file is located at the following url: <http://www.csub.edu/~emontoya2/datasets/mlb2010.csv>

To import the data into RStudio, we use `read.csv()` and it is a simple function to use. It just requires the location of the file. If the data location is a URL, then use the url. If it is data that you’ve created or recorded and it is saved on your computer, then we use `file.choose()`. We will cover both methods here.

2.3.1 Importing data via a url

This only works if the .csv has a url location. The “MLB salaries (2010)” data set is located at the following url:

Link: <http://www.csub.edu/~emontoya2/datasets/mlb2010.csv>

To import, run

```
mlbDataImport = read.csv( "http://www.csub.edu/~emontoya2/datasets/mlb2010.csv" )
```

The `mlbDataImport` dataframe should now appear under the “Environment tab”.

Important: The url must be in quotations and NO spaces at the start after the quotation or right before the end of the quotation. For example, you will get an error if you use the following urls:

- " http://www.csub.edu/~emontoya2/datasets/mlb2010.csv"
- "http://www.csub.edu/~emontoya2/datasets/mlb2010.csv "
- " http://www.csub.edu/~emontoya2/datasets/mlb2010.csv "
- They look similar to what we used but it has spaces within the quotation marks!

2.3.2 Importing data from your computer

Download the `.csv` file below onto your computer. Make sure to note where your file is saved. Depending on your browser settings, after clicking on the link below, the file will either open in your browser or be downloaded.

<http://www.csub.edu/~emontoya2/datasets/sampledata.csv>

After saving this `.csv` file, run the following

```
### by putting in "file.choose()", this tells R that you want
### to browse for and select the csv file
ImportFromDrive = read.csv( file.choose() )
```

This automatically opens a window that allows you to browse for your `.csv` file. Select your file and click on *Open*. You should see the following in the console



```
> ImportFromDrive = read.csv( file.choose() )
```

Figure 2.3: Figure 2.3

The data set `ImportFromDrive` should appear in the *Environment* tab in the upper right panel where it displays the number of observations and variables.

Note: Data that you import will not have help files. Only datasets that come with R (or from R packages) have help files.

2.4 Using RStudio’s data import feature

Alternatively, one can take a “point and click” approach to importing data. Like most software, RStudio has a toolbar where you can access many commands. To import data using the toolbar menu, select *File>Import Dataset> From Text (base)*. Then select your *.csv* file. If your data is stored in an excel spreadsheet (*.xlsx*), then select “From Excel”.

2.5 Summarizing data

There are many ways to summarize data in **R**. We will be using functions from the *mosaic* package. From now on, always load the *mosaic* package. That is, always have the following in your R script before writing any commands to conduct any analysis:

```
require( mosaic )
library(lattice)
```

When wanting to summarize a single variable, the formula to create graphical and numerical summaries follows the general form `goal( ~ x , data = dataframe)` where *x* is the variable you want to graph or compute a statistic of. Depending on the summary, we may have to specify some additional things. If the variable *x* is a variable in the dataframe/dataset, then data would be set equal to the name of the data frame. The names used for *x* and the name of the dataframe must be the names that R has given the data set and variable. `goal` would be replaced with the name of the function.

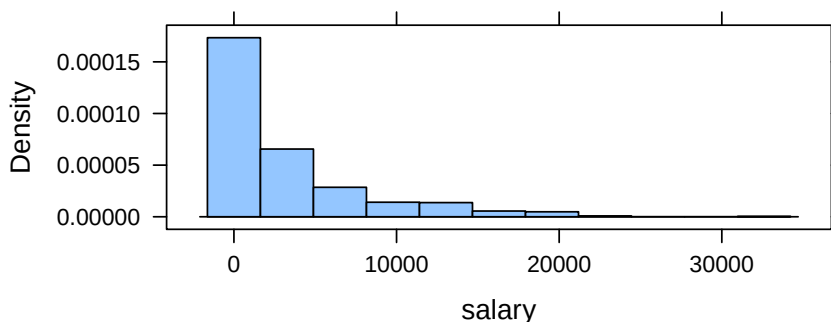
Recall that we imported the MLB dataset and we stored it in a dataframe called `mlbDataImport`. Let’s take a glimpse at the data

```
pillar::glimpse( mlbDataImport ) # pillar::glimpse() is provided by pillar
## Rows: 828
## Columns: 4
## $ player <chr> "Brandon Webb", "Danny Haren", "Chris Snyder", "Edwin Jackson~
## $ team <chr> "Arizona Diamondbacks", "Arizona Diamondbacks", "Arizona Diam~
## $ position <chr> "Pitcher", "Pitcher", "Catcher", "Pitcher", "First Baseman", ~
## $ salary <dbl> 8500.000, 8250.000, 5250.000, 4600.000, 4500.000, 4185.000, 3~
```

Let’s start with a simple graphical summary. Three widely used R plotting systems are *base*, *lattice*, and *ggplot2*. We will be using the *lattice* system, which comes from the *lattice* package. The *lattice* package comes with R. Load it explicitly with `library(lattice)` to use functions such as `histogram()` and `bwplot()`.

To start, we will create a histogram of `salary`. `salary` represents the salary (in \$1000s) of a given baseball player in 2010. Again, the basic formula is `goal( ~ x , data = dataframe)` where `goal` would be replaced with the name of the function to create the desired graph. **R** refers to histograms as `histogram()`:

```
histogram( ~ salary , data= mlbDataImport )
```



Let's compute the mean salary of these baseball players. Again, `salary` represents the salary of a given baseball player in 2010. As with the graph, the basic formula is `goal( ~ x , data = dataframe)` where `goal` would be replaced with the name of the function to compute the desired sample statistic. To compute the mean, **R** calls its function `mean`:

```
mean( ~ salary , data= mlbDataImport )
## [1] 3281.828
```

Suppose you wanted to compute the average salary for each position. To do so, we can tell RStudio to compute the mean by levels/groups of a grouping variable. A grouping variable identifies categories or groups. Grouping variables are categorical variables. When summaries are desired based on the levels or groups of the grouping variables, they are called side-by-side or comparative summaries.

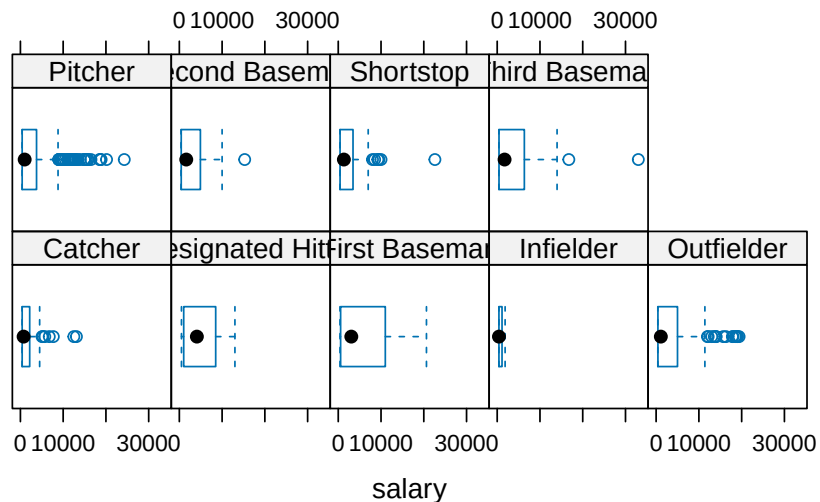
For any side-by-side or comparative data summary, the general formula changes a bit. The formula is `goal( ~ x | gfactor , data = dataframe)`, where `gfactor` represents the grouping variable. In our example, the grouping variable is `position`:

```
mean( ~ salary | position , data= mlbDataImport )
##           Catcher Designated Hitter      First Baseman      Infielder
##           1937.220           5235.714           5826.521           770.575
##           Outfielder           Pitcher      Second Baseman      Shortstop
```

```
##          3753.926          2999.197          3022.723          2844.146
##      Third Baseman
##          4641.286
```

One may also create a side-by-side or comparative graphical summary. To create a boxplot of `salary` for each position, run the following:

```
bwplot( ~ salary | position , data= mlbDataImport )
```



2.6 Export an R plot or R output

2.6.1 Exporting graphs

One may need to create a graph or one may have to submit *RStudio* output for certain assignments or exam problems. To export a plot, go to the *Plots* tab in the bottom right panel, click on *Export>...*

You have three export format options:

- “Save as Image...”
- “Save as PDF...”
- “Copy to Clipboard...”

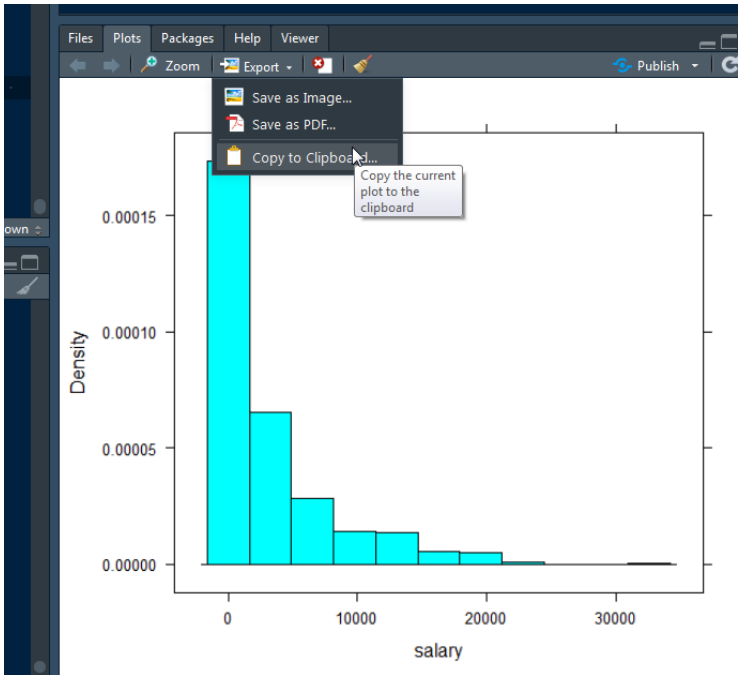


Figure 2.4: Figure 2.4

Select your preferred option. A small window should then appear of the plot. In the pop up window you may adjust the size of the plot and provide a name for the new file (see below for an example when one selects “Copy to Clipboard”).

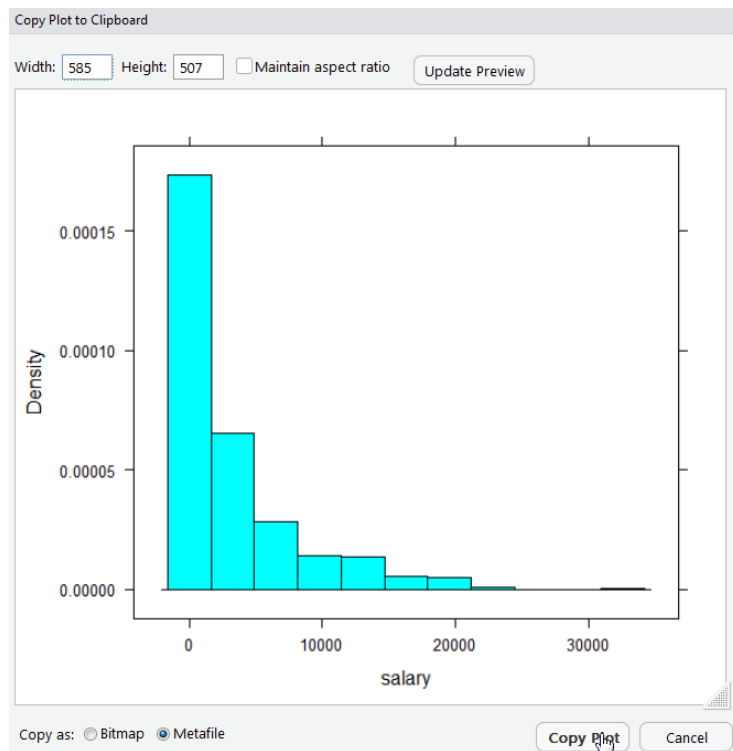


Figure 2.5: Figure 2.5

2.6.2 Exporting R output

If **R** output is requested, then highlight and copy the output from the **R** console. It may then be pasted in your Word or Google Doc document. Alternatively, take a screenshot of the desired output.

2.7 Extracting variables from dataframes

To extract a given variable from a dataframe, we place a **\$** at the end of the dataframe name, followed by the variable name. To illustrate, we use R’s built-in dataset called **trees**, which contains the diameter, height and volume for 31 cherry trees:

```
pillar::glimpse( trees )
## Rows: 31
## Columns: 3
## $ Girth <dbl> 8.3, 8.6, 8.8, 10.5, 10.7, 10.8, 11.0, 11.0, 11.1, 11.2, 11.3, ~
## $ Height <dbl> 70, 65, 63, 72, 81, 83, 66, 75, 80, 75, 79, 76, 76, 69, 75, 74, ~
## $ Volume <dbl> 10.3, 10.3, 10.2, 16.4, 18.8, 19.7, 15.6, 18.2, 22.6, 19.9, 24.~
```

To extract the variable `Volume`, do the following:

```
trees$Volume
## [1] 10.3 10.3 10.2 16.4 18.8 19.7 15.6 18.2 22.6 19.9 24.2 21.0 21.4 21.3 19.1
## [16] 22.2 33.8 27.4 25.7 24.9 34.5 31.7 36.3 38.3 42.6 55.4 55.7 58.3 51.5 51.0
## [31] 77.0

### Let's store the volume measurements
### into an R object called 'treeVol'.
### We can then call 'treeVol' whenever
### we need to use these measurements.
treeVol <- trees$Volume # assign vol. meas. to 'treeVol'
```

2.8 Understanding common R errors

Everyone gets errors in **R**. It's part of the process of learning any new software. Here are some of the most common errors you may encounter and what they mean

2.8.1 could not find function

Example:

```
> whatIsThis( 4 ) # no such function/command exists
Error in whatIsThis(4) : could not find function "whatIsThis"
```

This error usually occurs when

- you are using an **R** formula or function that does not exist.
- It exists, but you haven't loaded the *R* package that has this function/formula. Remember to always run `require( mosaic )` when you start **RStudio**.
- You misspelled the name of the function. For example, the first command works, but the second does not (**R** is case sensitive!)

```
mean( ~ salary , data= mlbDataImport )
## [1] 3281.828
```

```
> Mean( ~ salary , data= mlbDataImport )
Error in Mean(~salary, data = mlbDataImport) :
  could not find function "Mean"
```

2.8.2 object not found

Example:

```
> MlbDataImport
Error: object 'MlbDataImport' not found
```

This error usually occurs because the object has not been defined. You will also frequently see this when you’ve mistyped the name of the object you think you are referencing. Remember **R** is case sensitive! The name of the data set is `mlbDataImport` not `MlbDataImport`.

2.8.3 Unmatched parenthesis

Another common error is forgetting or neglecting to finish a call to a function/formula with a closing `)` or adding too many parentheses. An example of this follows:

```
> histogram( ~ salary , data= mlbDataImport ) ) )
Error: unexpected ')' in "histogram( ~ salary , data= mlbDataImport ) )"
```

If you forget a parenthesis, R will keep prompting you by returning `+`:

```
> histogram( ~ salary , data= mlbDataImport
+
+
+
+
### Hit the 'Esc' key a few times to have
### R stop prompting.
```

2.8.4 The ‘Some package’ package is not available (for R version ‘some version’)

This generally means that:

- A package named ‘Some package’ exists but it’s not available for your version of R (not a common error for this type of class)
- No package exists with the name ‘Some package’ (a more common error)

It is likely that you misspelled the name, were not careful with case-sensitivity, or inserted spaces at the start/end of the package name.

Please keep in mind that everyone makes errors when learning new software. Oftentimes, you will find that you are able to understand what they mean by carefully reading over them. When you can’t, carefully look over your R file and consider posting your question on the “Student-led RStudio Q&A” discussion board or scheduling a student hour appointment.

2.9 Help files

All built in functions, as well as data sets, in **R** have help files. To access the help files for a function or data set one uses the function `help( )`. In general, the help file will contain the following sections:

- Description: Gives brief description of the function.
- Usage: Displays a function’s arguments, along with the default values of certain arguments.
- Arguments: Gives details of each of the function’s arguments.
- Value: Describes what is returned by the function.
- See Also: Lists links to help files of similar functions.
- Examples: Gives example code that uses the function(s).

For example, the following commands bring up the help file on the `log( )` function,

```
> help(log)
Help for 'log' is shown in the browser
> ?log # This is equivalent to help(log)
Help for 'log' is shown in the browser
>
```

Either of these commands should open the following (abbreviated) help file

```
log {base}                                R Documentation

Logarithms and Exponentials Description

log computes natural logarithms, log10 computes common (i.e., base 10)
```

logarithms, and `log2` computes binary (i.e., base 2) logarithms. The general form `logb(x, base)` computes logarithms with base `base`.

Usage:

```
log(x, base = exp(1))
logb(x, base = exp(1))
log10(x)
log2(x)
```

Arguments:

`x` a numeric or complex vector.
`base` positive number. The base with respect to which logarithms are computed. Defaults to `e=exp(1)`.

Value:

A vector of the same length as `x` containing the transformed values. `log(0)` gives `-Inf` (when available).

...

Examples:

```
log(exp(3))
log10(1e7)# = 7
```

The function `log()` has two arguments. The first is a numeric vector and the second corresponds to the base. By default the function `log()` will use the natural logarithm (base `exp(1)`, or `e`) unless you specify otherwise as illustrated below.

```
> log(10)
[1] 2.302585
> log(10, base = exp(1))
[1] 2.302585
> log(10, base = 2)
[1] 3.321928
>
```

If you seek an example of using a given function, it is a good idea to look at the examples given at the end of the help file. Now suppose you had no idea that a logarithm function existed, but you would like to know if such a function might exist in **R**. To explore this question you can issue `help.search( "logarithm"`

) which will search the help files for documentation matching the character string `logarithm`. You may also run `RSiteSearch( "logarithm" )` which will search for the character string “logarithm” in the R-help mailing list archives. Lastly, you can do a web search for your error as it is almost certain someone has had the same or similar error.

Chapter 3

R code for Regression Analysis

The goal of this chapter is to have you learn to use **R** to carry out the analyses and techniques generally covered in a regression analysis course. Currently, we carry out some of the examples found in

Kutner M.H, Nachtsheim C.J, and Neter, J., *Applied Linear Regression Models*. McGraw-Hill/Irwin.

The abbreviation ALRM is used for this textbook. Datasets for this textbook may be found at

<https://www.csub.edu/~emontoya2/datasets/textbookdata/index.html>

3.1 Linear Regression with One Predictor Variable

In this section, we use R to fit the simple linear regression (SLR) model

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

where y_i is the dependent/response variable, x_i is the independent/predictor variable, and the random error terms have mean 0, constant variance σ^2 and are uncorrelated. We follow the Toluca Company example given in ALRM to illustrate how to use **R** to obtain scatter plots, the least-squares estimates, fitted values, residuals, and a point estimator of σ^2 .

3.1.1 Scatterplots

Scatterplots can be used to explore the relationship between two variables. Recall that the formula to create a graphical numerical summary follows the general form `goal( ~ x , data)` where `x` is the variable you want to graph. However, now we have an additional variable. When the data is bivariate the general formula changes a bit: `goal( y ~ x , data )`, where `y` is the name of the response/dependent variable and `x` is the name of the explanatory/predictor variable. To create a scatterplot, `goal` is replaced with `xyplot`.

We begin by importing the data:

```
require( mosaic ) # always load the mosaic package first.
### The datasets from the book are available to download or
### read from a url, but we have to make some changes
### to read.csv() because these files are not csv files.

### Import data:
url <- "http://www.csub.edu/~emontoya2/datasets/textbookdata/Kutner/Chapter%20%201%20D"

### Use this as a template for importing datasets referenced
### in the textbook.

### This is not a .csv file and the file does not
### include variable names, so some changes
### have to be made to read.csv().
toluca.data <- read.csv( url , header = FALSE , col.names = c("xvar", "yvar") , sep = "
```

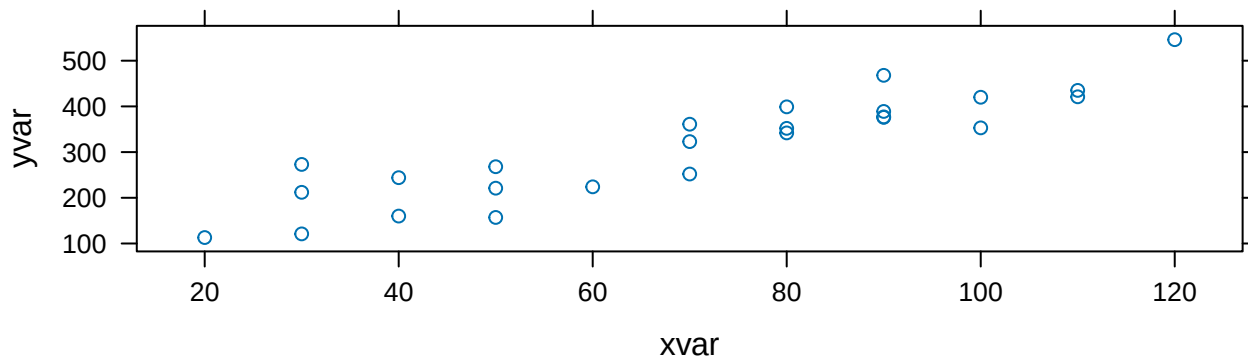
We can take a quick look at the data by using `pillar::glimpse( )`:

```
pillar::glimpse( toluca.data )

## Rows: 25
## Columns: 2
## $ xvar <int> 80, 30, 50, 90, 70, 60, 120, 80, 100, 50, 40, 70, 90, 20, 110, 10~
## $ yvar <int> 399, 121, 221, 376, 361, 224, 546, 352, 353, 157, 160, 252, 389, ~
```

Note that we named the response variable and explanatory variable `yvar` and `xvar`, respectively. So these names must be used in `xyplot( )` or any other function that uses `toluca.data`:

```
xyplot( yvar ~ xvar , data= toluca.data)
```



The function `xyplot( )` has other arguments that allow you to change axis labels, add a title, etc.:

```
### xyplot( y ~ x | gfactor, data, main, xlab, ylab, col, pch)
# y: response variable
# x: explanatory variable
# gfactor: a factor variable (optional) so that a plot is returned for each level of gfactor (optional)
# data: the name of the dataframe where the observed data is found
# main: title for plot (optional)
# ylab: label for y-axis (optional)
# xlab: label for x-axis (optional)
# pch: point symbol. Common numeric plotting symbols are 0-25 (optional)
```

3.1.2 Least Squares (LS) Estimates

To obtain the LS estimates, we can either compute them using the algebraic formulas or fit the SLR model using the function `lm( )`. The `lm( )` function follows the general form of `goal( y ~ x , data )`, where `goal` is replaced with `lm`. Note that `lm` stands for linear model.

To illustrate the first method, recall that the LS estimators of β_0 and β_1 are

$$b_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad b_0 = \bar{y} - b_1 \bar{x}$$

To apply these formulas, we use the function `mean( )`. This function follows the general form `goal( ~ x , data)` but `goal` becomes `mean` :

```

### extract 'xvar' and assign the values to `myx':
myx <- toluca.data$xvar
### same for yvar:
myy <- toluca.data$yvar

### Compute required differences.
### Note that we have no data argument
### since 'myx' was defined above and is
### not part of a dataframe.
XminusXbar <- myx - mean( ~ myx )
YminusYbar <- myy - mean( ~ myy )

XminusXbarSq <- XminusXbar^2 # squared differences

### Apply the LS formulas for LS estimates
b.1=sum( ( myx - mean( ~ myx ) )*( myy - mean( ~ myy ) ) )/sum( ( myx-mean( ~ myx ) )^2 )
b.1

## [1] 3.570202

b.0=mean( ~ myy ) - b.1*mean( ~ myx )
b.0

```

```
## [1] 62.36586
```

Thus, $b_1 = 3.570$ and $b_0 = 62.366$.

The `lm( )` function follows the formula `goal( y ~ x , data )`. This function requires the name of the dataframe and the variables used in the dataframe. Recall that the variables in `toluca.data` are called `yvar` and `xvar`:

```

### fit the model
lm( yvar ~ xvar , data= toluca.data )

##
## Call:
## lm(formula = yvar ~ xvar, data = toluca.data)
##
## Coefficients:
## (Intercept)      xvar
##      62.37      3.57

```

Although the `lm( )` function only prints the LS estimates, it actually computes a lot of information that is only retrievable if we store the `lm( )` result in an

R object. Such objects are generally called *lm* objects. So, we store the result of `lm()` to an *R* object (I called it `toluca.fit` below) so that we can extract certain information as needed:

```
### fit the model and store the model fit
toluca.fit <- lm( yvar ~ xvar , data= toluca.data)
```

To view the LS estimates, we summarize the *lm* object, `toluca.fit`, using the `summary()` function:

```
### summarize the fit <-- returns a lot of information
summary( toluca.fit )

##
## Call:
## lm(formula = yvar ~ xvar, data = toluca.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -83.876 -34.088  -5.982  38.826 103.528
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   62.366     26.177   2.382  0.0259 *
## xvar           3.570      0.347  10.290 4.45e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 48.82 on 23 degrees of freedom
## Multiple R-squared:  0.8215, Adjusted R-squared:  0.8138
## F-statistic: 105.9 on 1 and 23 DF,  p-value: 4.449e-10
```

From the `summary()` output, we see that $b_0 = 62.366$ and $b_1 = 3.570$. More on the output provided by `summary()` later.

Alternatively, we can extract the LS estimates by applying the function `coef()` to the *lm* object:

```
### The only info required is the name
### of your lm object.
coef( toluca.fit )
```

```
## (Intercept)      xvar
##  62.365859    3.570202
```

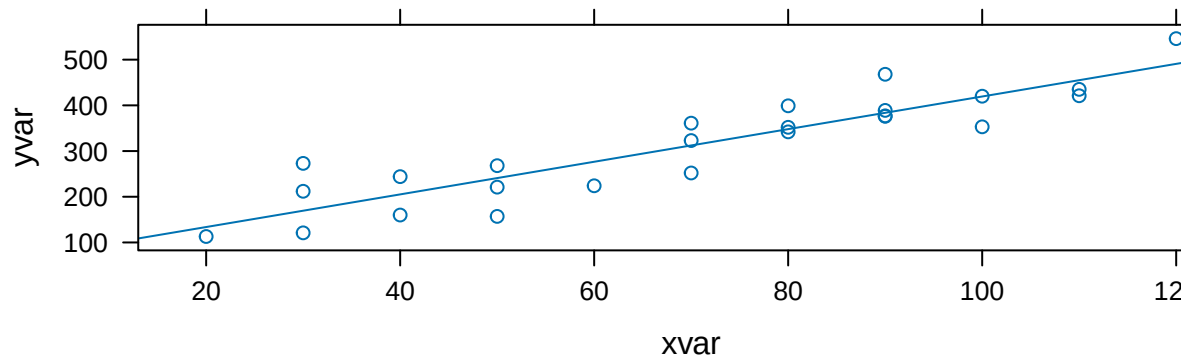
3.1.3 Plotting the estimated regression line

Once we have access to our data or we fit and save the simple linear regression model using `lm()`, we can plot the data along with the estimated LS line. There are two approaches for plotting the LS line: `xyplot()` or `plotModel()`.

3.1.3.1 Using `xyplot()`

The function `xyplot()` is used the same way as before but with an additional argument of `type= c( "p", "r" )`:

```
### Plot the data and model fit.
### Note: "p" tells R we want to plot the points.
###      "r" tells R we want to add the est. regression
###      line to the plot
xyplot( yvar ~ xvar , data= toluca.data , type= c( "p", "r" ) )
```

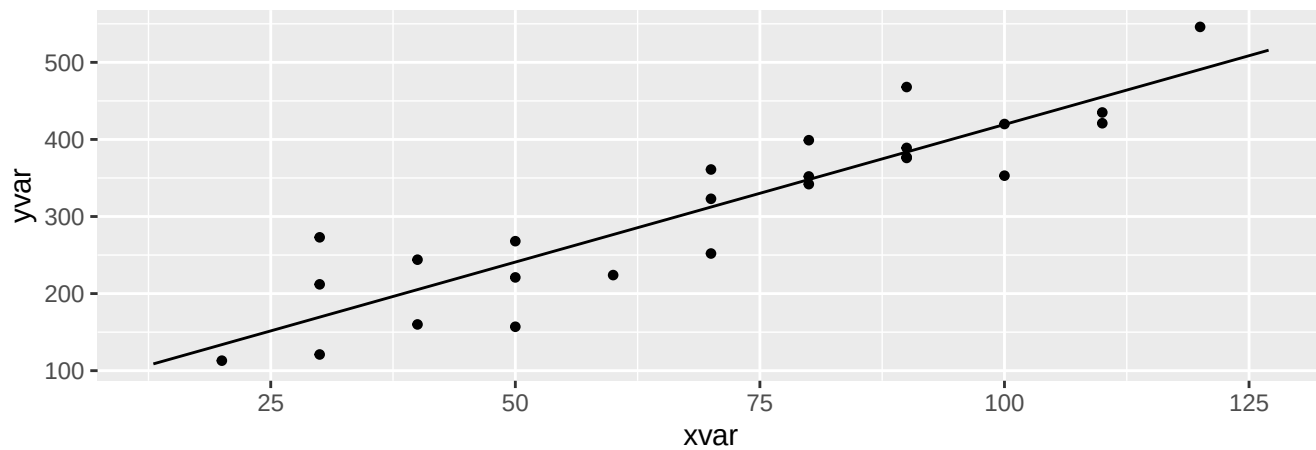


The argument `type= c( "p", "r" )` tells **R** to plot the points and the estimated regression line. Also recall that `xyplot()` has additional arguments to change axis label, add a title, etc.

3.1.3.2 Using `plotModel()`

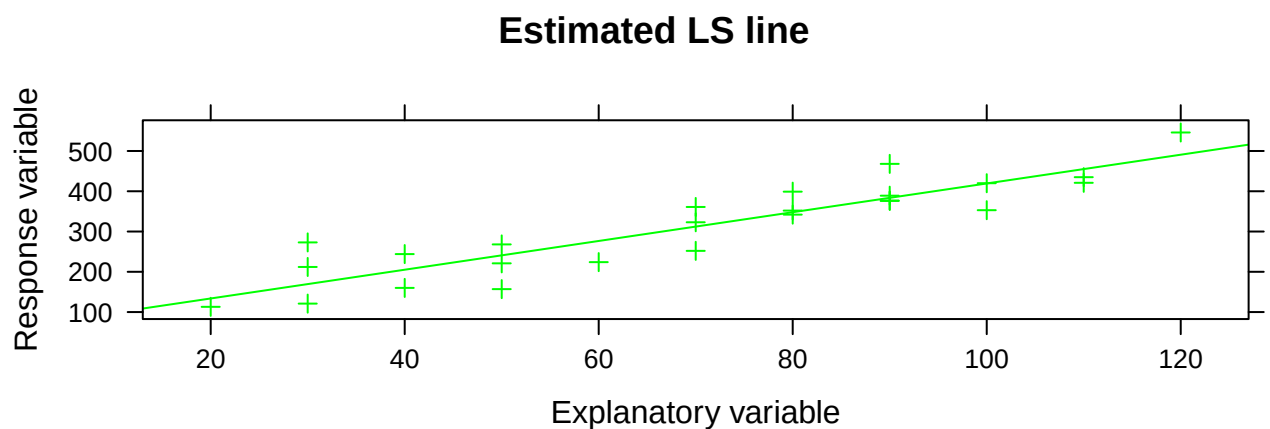
`plotModel()` will plot the data and the estimated LS line. For this example, its required argument is an *lm object*:

```
### Plot the data and model fit.
plotModel( toluca.fit )
```



Both functions can display the fitted line. Use `xyplot()` when you want to set lattice plot options explicitly, as in the following example:

```
xyplot( yvar ~ xvar , data = toluca.data , type= c( "p", "r" ), xlab = "Explanatory variable", ylab = "Response variable"
```



3.1.4 Fitted or predicted values

To obtain the fitted values, $\hat{y}_i = b_0 + b_1 x_i$ for $i = 1, 2, \dots, n$, we may use **R** by directly using the least-squares estimates obtained by using the summation formulas or more conveniently use the `fitted.values( )` or `predict( )` on an *lm* object:

```
### Only one argument: An lm object
fitted.values( toluca.fit )
```

```
##      1      2      3      4      5      6      7      8
## 347.9820 169.4719 240.8760 383.6840 312.2800 276.5780 490.7901 347.9820
##      9     10     11     12     13     14     15     16
## 419.3861 240.8760 205.1739 312.2800 383.6840 133.7699 455.0881 419.3861
##     17     18     19     20     21     22     23     24
## 169.4719 240.8760 383.6840 455.0881 169.4719 383.6840 205.1739 347.9820
##      25
## 312.2800
```

```
### predict( )
predict( toluca.fit )
```

```
##      1      2      3      4      5      6      7      8
## 347.9820 169.4719 240.8760 383.6840 312.2800 276.5780 490.7901 347.9820
##      9     10     11     12     13     14     15     16
## 419.3861 240.8760 205.1739 312.2800 383.6840 133.7699 455.0881 419.3861
##     17     18     19     20     21     22     23     24
## 169.4719 240.8760 383.6840 455.0881 169.4719 383.6840 205.1739 347.9820
##      25
## 312.2800
```

Both `fitted.values( )` and `predict( )` can provide the fitted/predicted values for the observed values of x . If instead you wanted to predict or fit values of y for certain levels or values of x , we use `predict( )` with an additional argument that corresponds to a dataframe that holds the x value(s) of interest. For example:

```
### This is a template on how to predict values of y
### for certain values of x.

### Suppose we wanted to predict the response when x=31 or x=119.
### First set up the dataframe that holds these values. Note that
### we have to use the name 'xvar' since this is the name of
### the predictor in toluca.data.
```

```
newdat <- data.frame( xvar=c(31, 119) ) # 'xvar' is the name of
                                         # explanatory variable in the dataframe

### You have to specify an 'lm object'.
### newdata is a data frame that holds the x values of interest.
predict( toluca.fit , newdata= newdat )

##          1          2
## 173.0421 487.2199
```

3.1.5 Residuals

The residuals are the differences between the observed values and the fitted values, denoted by $e_i = y_i - \hat{y}_i$. The residuals can be obtained using the function `residuals( )` with an *lm object* as its first argument:

```
### You have to specify an 'lm object'.
residuals( toluca.fit )

##          1          2          3          4          5          6
## 51.0179798 -48.4719192 -19.8759596 -7.6840404 48.7200000 -52.5779798
##          7          8          9         10         11         12
## 55.2098990  4.0179798 -66.3860606 -83.8759596 -45.1739394 -60.2800000
##          13         14         15         16         17         18
##  5.3159596 -20.7698990 -20.0880808  0.6139394 42.5280808 27.1240404
##          19         20         21         22         23         24
## -6.6840404 -34.0880808 103.5280808 84.3159596 38.8260606 -5.9820202
##          25
## 10.7200000
```

```
e <- residuals( toluca.fit ) # store them in an object called 'e'
```

You may also compute them via:

```
### You have to specify an 'lm object'.
yhat <- fitted.values( toluca.fit )

toluca.data$yvar - yhat # observed - fitted/predicted

##          1          2          3          4          5          6
## 51.0179798 -48.4719192 -19.8759596 -7.6840404 48.7200000 -52.5779798
##          7          8          9         10         11         12
```

```
## 55.2098990 4.0179798 -66.3860606 -83.8759596 -45.1739394 -60.2800000
##          13          14          15          16          17          18
## 5.3159596 -20.7698990 -20.0880808 0.6139394 42.5280808 27.1240404
##          19          20          21          22          23          24
## -6.6840404 -34.0880808 103.5280808 84.3159596 38.8260606 -5.9820202
##          25
## 10.7200000
```

```
e <- toluca.data$yvar - yhat # store the result
```

3.1.6 `summary( lm object )` output

So what is printed by `summary( lm object )`? The following output is given:

Coefficients:

	Estimate	Std. Error	t value	$Pr(> t)$
(Intercept)	b_0	$s\{b_0\}$	$t_{b_0}^* = \frac{b_0}{s\{b_0\}}$	$2P(t_{n-2} > t_{b_0}^*)$
X	b_1	$s\{b_1\}$	$t_{b_1}^* = \frac{b_1}{s\{b_1\}}$	$2P(t_{n-2} > t_{b_1}^*)$

Residual standard error: $\sqrt{MSE}$ on $n - 2$ degrees of freedom

Multiple R-Squared: R^2 , Adjusted R-squared: $R_{adjusted}^2$

F-statistic: $F^* = \frac{MSR}{MSE}$ on 1 and $n - 2$ DF, p-value: $P(F_{1,n-2} > F^*)$

The

`summary( )` output provides the LS estimates, the standard errors of the LS estimates, the test statistic for the regression line parameters, two-sided p-values for these parameters, the residual standard error ($\sqrt{MSE}$), R^2 , and other information that we will revisit later.

3.1.7 Estimating σ^2

To estimate the error variance component, one can either use the formula or use the `summary( )` output. Using the formula we get:

```
### Recall we defined the residuals in a previous section
n <- 25 # from glimpse
sum( e^2 ) / ( n - 2 )
```

```
## [1] 2383.716
```

The summary output provides $\sqrt{MSE}$, which is an estimate of σ . Square the residual standard error to obtain MSE. Using the unrounded fit, $\hat{\sigma}^2 = 2383.716$.

3.2 Inference in simple linear regression

In this section, inference of the regression parameters using confidence intervals and hypothesis testing, inference about the mean response, prediction intervals for new observations, the ANOVA approach, and measures of association are addressed using **R**.

3.2.1 Inferences for β_0 and β_1

The Toluca Company example from ALRM is used to illustrate inference on the slope and intercept of the model. Generally, inference is made about the slope of the model. A test and confidence interval concerning the intercept can be set up in the same manner as that of β_1 . To obtain the test statistic, along with p-values for tests regarding those parameters, we summarize the *lm object* (the model fit) by using `summary()`:

```
summary( toluca.fit)
##
## Call:
## lm(formula = yvar ~ xvar, data = toluca.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -83.876 -34.088  -5.982  38.826 103.528
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   62.366     26.177   2.382  0.0259 *
## xvar           3.570       0.347  10.290 4.45e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 48.82 on 23 degrees of freedom
## Multiple R-squared:  0.8215, Adjusted R-squared:  0.8138
## F-statistic: 105.9 on 1 and 23 DF,  p-value: 4.449e-10
```

Coefficients:

	Estimate	Std. Error	t value	$Pr(> t)$
(Intercept)	b_0	$s\{b_0\}$	$t_{b_0}^* = \frac{b_0}{s\{b_0\}}$	$2P(t_{n-2} > t_{b_0}^*)$
X	b_1	$s\{b_1\}$	$t_{b_1}^* = \frac{b_1}{s\{b_1\}}$	$2P(t_{n-2} > t_{b_1}^*)$

Residual standard error: $\sqrt{MSE}$ on $n - 2$ degrees of freedom

Multiple R-Squared: R^2 , Adjusted R-squared: $R_{adjusted}^2$

F-statistic: $F^* = \frac{MSR}{MSE}$ on 1 and $n - 2$ DF, p-value: $P(F_{1,n-2} > F^*)$

Recall that the `summary()` output provides the following:

Based on the output of `summary( toluca.fit )`, we obtain the estimates of β_1 and β_0 along with the test statistics $b_1/s\{b_1\}$ and $b_0/s\{b_0\}$ with the corresponding two-sided p-value for each.

Alternatively, we can also obtain the test statistic using the following commands:

```
MSE <- sum( e^2 )/( n-2 )

sd.b1 <-sqrt( MSE / ( sum( (myx -mean( ~ myx ) )^2 ) ) )

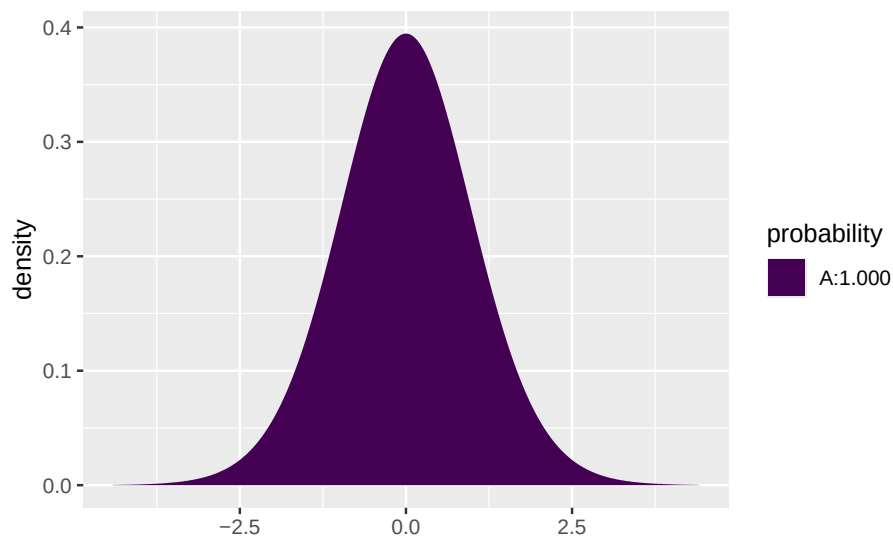
t.star= b.1/sd.b1
t.star
## [1] 10.28959

# The test for the intercept is computed similarly
```

Under $H_0 : \beta_1 = 0$, and assuming independent normal errors with constant variance, the statistic has a t distribution with $n-2$ degrees of freedom (notation: t_{n-2}). Recall that the p-value is the probability that the test statistic would take a value as extreme (or more extreme) as the observed test statistic in the direction of the alternative if H_0 were true.

If $H_a : \beta_1 > 0$, the p-value = $P(t_{n-2} > 10.290)$. To obtain this probability we use the function `xpt( q, df, lower.tail )`. The value `q` will be the value of interest (10.290 in this example), `df` corresponds to the degrees of freedom ($n-2$), and `lower.tail` will either be set equal to `TRUE` (computes the area to the left of `q`) or `FALSE` (computes the area to the right of `q`). To compute $P(t_{n-2} > 10.290)$, set `lower.tail=FALSE`:

```
xpt( q= 10.290, df= 25 - 2, lower.tail =FALSE)
```

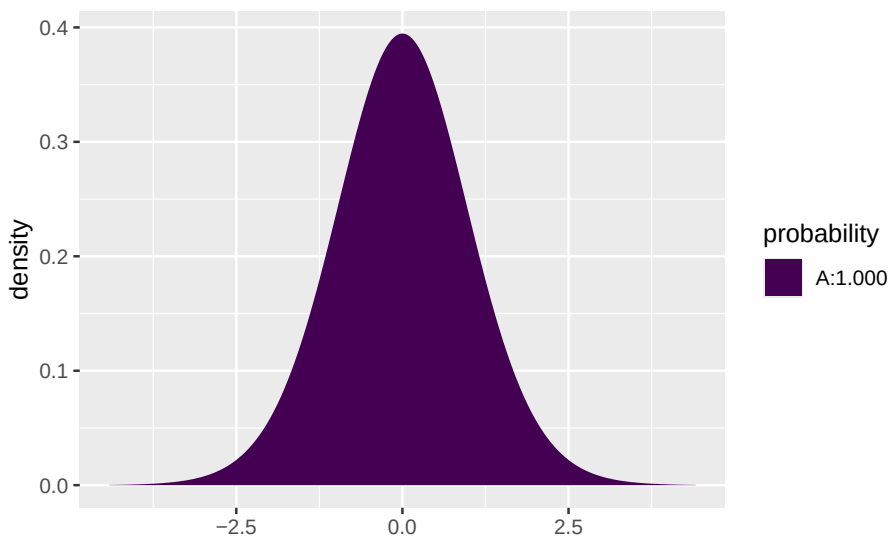


```
## [1] 2.222735e-10
```

The desired probability will be printed in the console: $P(t_{n-2} > 10.290) \approx .0000000002$. This function will also produce a graph of the probability distribution with the area to the left of q shaded one color (area A) and the area to the right of q shaded another color (area B).

If instead $H_a : \beta_1 < 0$, then set `lower.tail=TRUE`:

```
xpt( q= 10.290, df= 25 - 2, lower.tail =TRUE)
```



```
## [1] 1
```

The output shows that $P(t_{n-2} < 10.290) \approx 1$. Lastly, if $H_a : \beta_1 \neq 0$, then we care about both possible extremes:

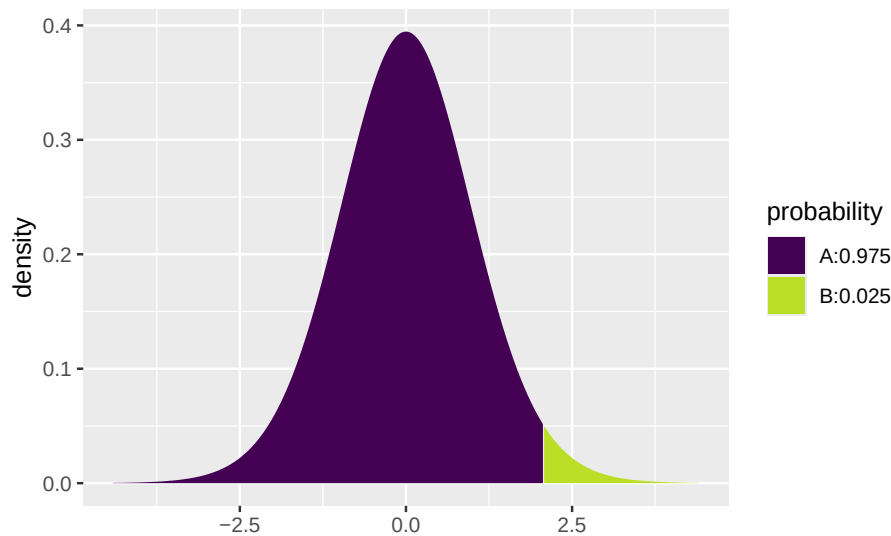
$$P(t_{n-2} < -10.290) + P(t_{n-2} > 10.290) = 2 \times P(t_{n-2} > |10.290|) = .0000000004$$

For a confidence interval (CI) for β_1 , recall that the limits of a $1 - \alpha$ confidence interval for β_1 are

$$b_1 \pm t_{1-\alpha/2, n-2} s\{b_1\}$$

To derive the CI, we can either use the `summary()` output along with the critical values provided by `xqt()`, or you can use `confint()`. Let's first derive the CI using `xqt(p, df, lower.tail)`. This function provides the quantile for a specified probability. For the Toluca Company example suppose a 95% confidence interval is to be computed. Then, $t_{1-.05/2, 25-2} = \text{xqt}(1-.05/2, \text{df}= 25-2) = 2.069$.

```
xqt( 1-.05/2, df= 25-2)
```



```
## [1] 2.068658
```

To obtain the CI, run the code below:

```
b1 <- 3.570 # from summary()
sb1 <- 0.347 # from summary()

b1 - 2.069*.347
## [1] 2.852057
b1 + 2.069*.347
## [1] 4.287943
```

Since the model has been fitted, `confint()` will provide the CIs for both parameters. This function has two arguments: an *lm object* and the desired confidence level:

```
confint( toluca.fit, level = .95)
##           2.5 %      97.5 %
## (Intercept) 8.213711 116.518006
## xvar       2.852435   4.287969
```

3.2.2 CI for the mean response ($E(y_h)$) and a prediction interval for y_h

Following the Toluca Company example from ALRM, we obtain a CI for the mean response $E(Y_h)$ at $x_h = 100$ units by using the function `predict()`.

However, the function has two additional arguments, `interval` and `level`. We set `interval = "confidence"` and `level` to the desired confidence level:

```
X.new<- data.frame(xvar = c( 100 ) ) # xvar refers to the name of the variable
predict.lm( toluca.fit , newdata= X.new , interval = "confidence", level= .90 )
```

```
##          fit          lwr          upr
## 1 419.3861 394.9251 443.847
```

The prediction of a new observation and its corresponding prediction interval (PI) can be obtained in the same manner as the confidence interval for the mean response but we set `interval = "prediction"`:

```
### Note: X.new was defined above
predict.lm( toluca.fit , newdata= X.new , interval= "prediction", level= .90 )
```

```
##          fit          lwr          upr
## 1 419.3861 332.2072 506.5649
```

3.2.3 Analysis of Variance approach to regression analysis

To obtain the ANOVA table in **R**, we use the function `anova( lm object )`:

```
anova(toluca.fit)
```

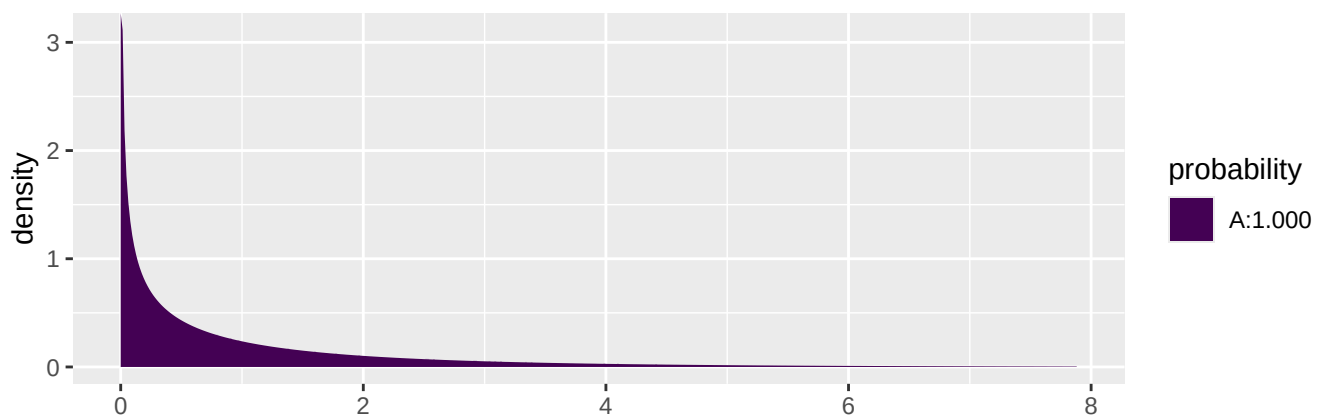
```
## Analysis of Variance Table
##
## Response: yvar
##          Df Sum Sq Mean Sq F value    Pr(>F)
## xvar         1 252378   252378  105.88 4.449e-10 ***
## Residuals   23   54825     2384
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

AOV Table				
Response: Y				
	Df	Sum Sq	Mean Sq	F value
X	1	SSR	MSR	$F^* = \text{MSR}/\text{MSE}$
Residuals	$n - 2$	SSE	MSE	

The following output is given by `anova( lm object )`:

The output from `anova( toluca.fit )` shows that the p-value ≈ 0 . We could also use the function `xpf( q , df1, df2, lower.tail )` to compute the p-value. The value `q` is the value of interest (105.88 in this example), `df1` corresponds to the numerator degree of freedom (1), `df2` corresponds to the denominator degree of freedom (25-2), and `lower.tail` would be set to `FALSE` since we want the p-value for this F-test:

```
xpf(q= 105.88, df1= 1, df2= 25-2 , lower.tail= FALSE)
```



```
## [1] 4.44711e-10
```

Note that the shaded “A” area in the graph always corresponds to the area to the left of the value of interest. Area “B” does not appear since it is too small (≈ 0)

3.2.4 Measures of association

The coefficient of determination (R^2) and coefficient of correlation (r) can be obtained from the SSR and SSTO (from the ANOVA table), by applying the summation formulas, or obtained from the `summary( lm object )` output. The coefficient of correlation is $\text{sign}(b_1)(\sqrt{R^2})$. Using the summation formulas:

```
### extract the response and predictor
yresp <- toluca.data$yvar
xpred <- toluca.data$xvar

b.1 # recall the value of the slope est.
```

```
## [1] 3.570202
```

```
SSR = b.1^2 * sum( ( xpred - mean( xpred ) )^2 )
SST0 = sum( ( yresp - mean( yresp ) )^2 )
R.sq = SSR/SST0
R.sq
```

```
## [1] 0.8215335
```

```
r=(1)*sqrt(R.sq) # slope est. is positive
```

Note that this matches the information provided in the output of summary discussed in a previous section.

3.3 Residual analysis and remedial measures

This section deals with assessing the appropriateness of the simple regression model. Residual analyses are performed on the Toluca Company example from ALRM.

If we assume that $\varepsilon_i \stackrel{iid}{\sim} N(0, \sigma^2)$ in the normal simple linear regression model, then residual plots can help assess these error assumptions. The fitted residuals are correlated and have variances $\sigma^2(1 - h_{ii})$; they are not themselves independent and identically distributed errors. We examine the assumption using the following residual plots:

Things to examine	Residual plots to examine
Normality	Normal QQ-plot of the residual
Independence	residuals vs. time or observation order, when meaningful
Constant variance	residuals vs x or $\hat{y}$
Linearity	residuals vs x or $\hat{y}$
Outliers	residuals vs x or $\hat{y}$; QQ-plot of the residuals

A normal Q-Q plot can reveal departures from normality, including heavy tails. Independence also requires consideration of how the data were collected; residuals versus a predictor alone cannot establish it.

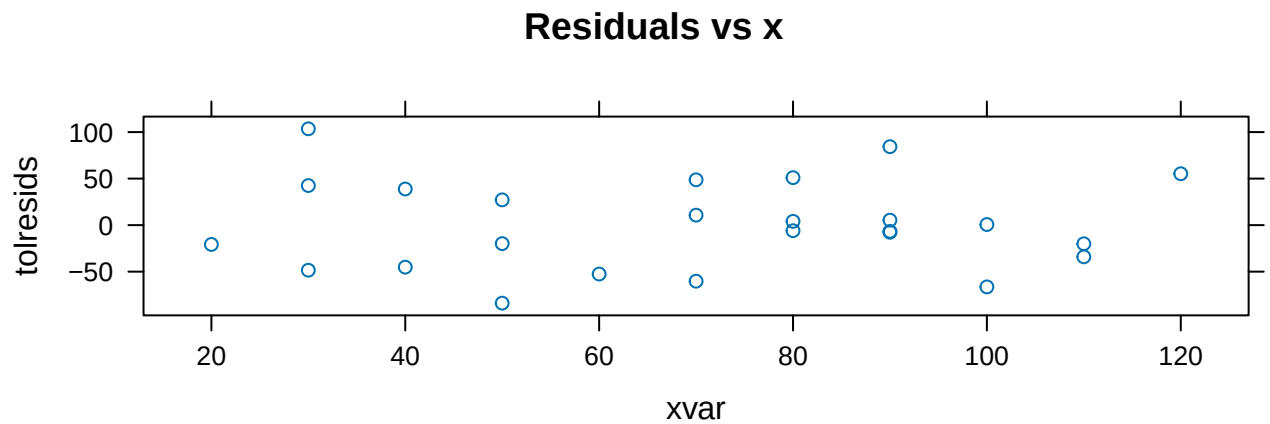
3.3.1 Creating residual plots

To create the plots to assess each of these assumptions, we use `xyplot()` and `xqqmath()`. The following code creates the required plots:

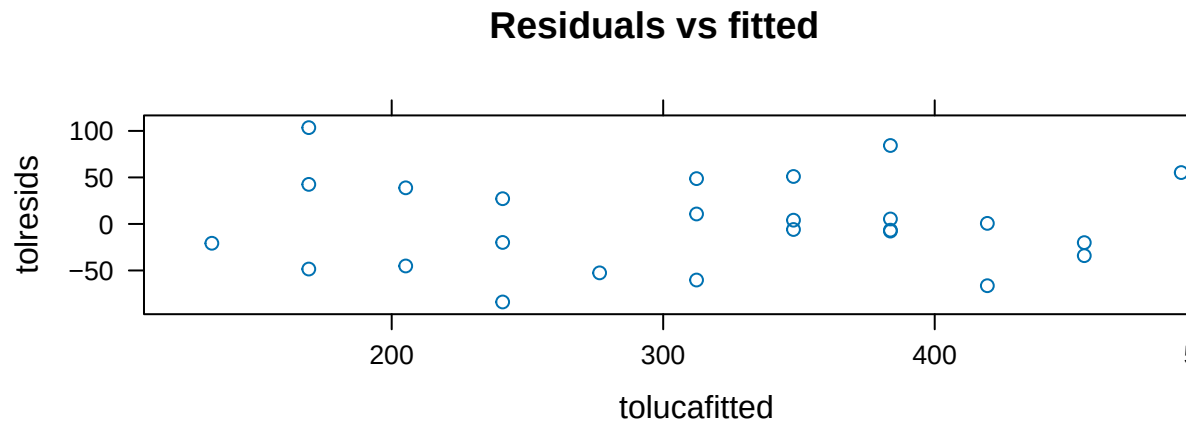
```
### extract the fitted values from the lm object
tolucafitted <- fitted( toluca.fit )

### extract the residuals from the lm object
tolresids <- residuals( toluca.fit )

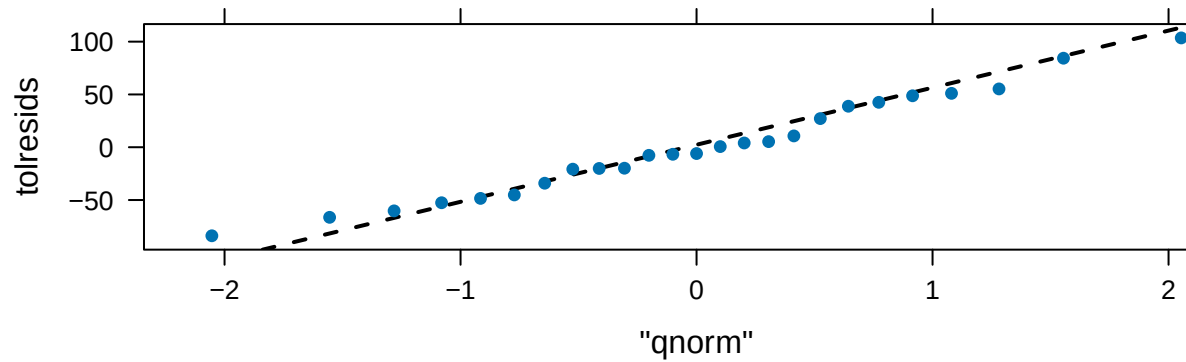
### Note: There is no time variable
### Residuals vs x
xyplot( tolresids ~ xvar, data=toluca.data, main= "Residuals vs x")
```



```
### Residuals vs fitted values
xyplot( tolresids ~ tolucafitted, data=toluca.data, main= "Residuals vs fitted")
```

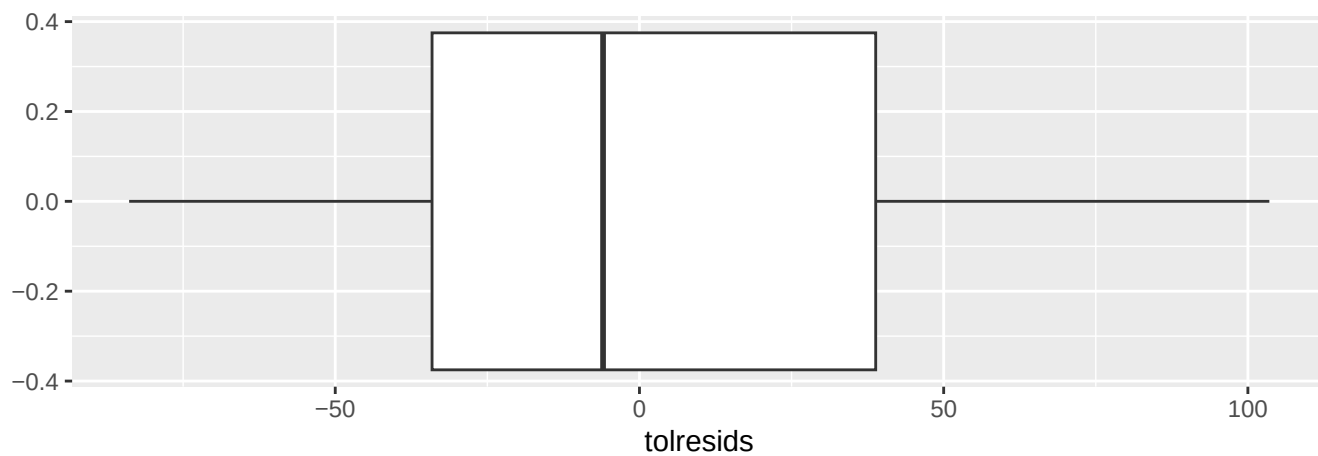


```
### QQ-plot of the residuals.
### xqqmath follows the general formula
### with an additional argument: goal( ~ x, data, distribution= "qnorm")
### There is no data argument since the residuals are
### already in R's memory since we created
### 'tolresids'.
xqqmath( ~ tolresids, distribution = "qnorm" )
```

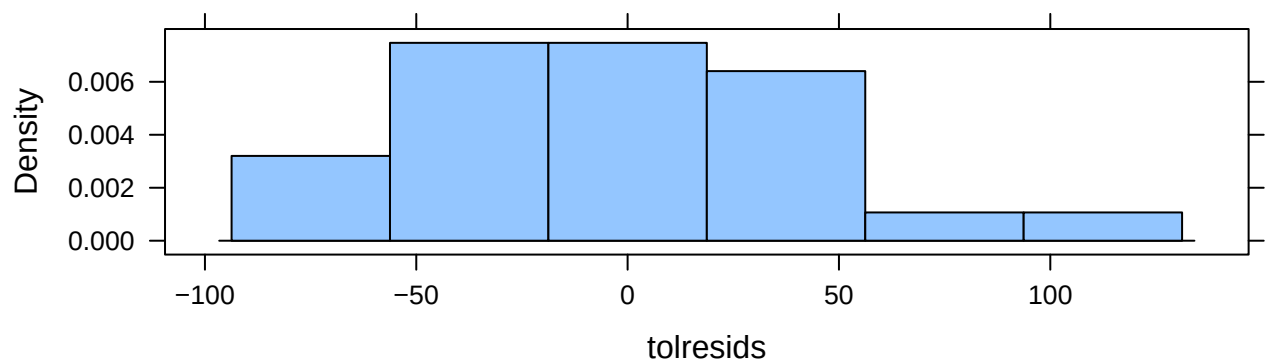


Although the above graphical summaries are generally sufficient for diagnostics, one may examine other residual plots:

```
### boxplot of the residuals  
gf_boxplot( ~ tolresids, main="Boxplot of residuals" )
```



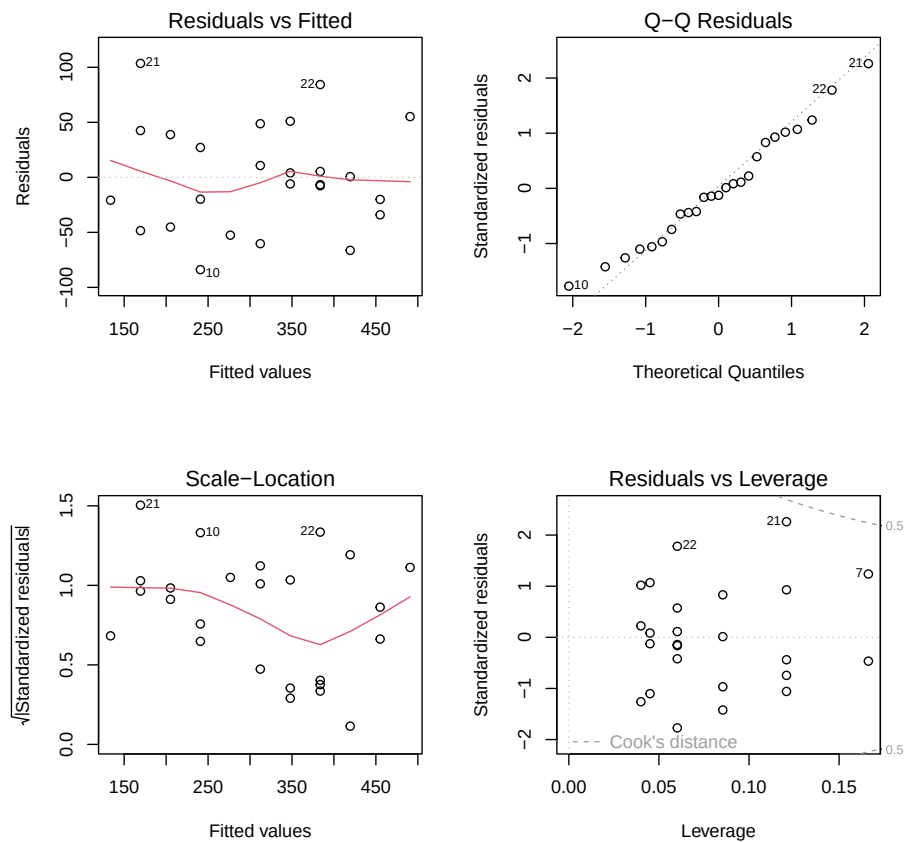
```
### histogram of the residuals  
histogram( ~ tolresids )
```



3.3.2 Residual diagnostic plots from base R

The `plot()` method for an *lm* object provides common diagnostic plots without an additional package:

```
oldpar <- par(mfrow = c(2, 2))
plot(toluca.fit, which = c(1, 2, 3, 5), ask = FALSE)
```



```
par(oldpar)
```

- **Residuals vs. fitted values:** Look for curvature and changes in spread, which can suggest an inadequate mean model or nonconstant variance.
- **Normal Q-Q plot:** Approximately straight points support the normal-error approximation; substantial departures warrant investigation.
- **Scale-location plot:** This plots the square root of the absolute standardized residuals against fitted values. A roughly constant spread supports the constant-variance assumption.
- **Residuals vs. leverage:** High leverage reflects unusual predictor values. Cases with high leverage and large residuals may be influential; Cook's-distance contours help identify them.

These plots suggest potential problems; they do not prove that the assumptions hold. See R's diagnostic-plot documentation.

3.3.3 Testing departures from normality

R code is provided to carry out the Shapiro-Wilk and Lilliefors (Kolmogorov-Smirnov) test for normality. Both tests may be used to detect if the residuals do not follow a normal distribution. The null hypothesis is that the residuals are normally distributed. The alternative is that they are not normally distributed. Thus, we are not testing if the residuals are normally distributed but rather if they depart from the normal distribution.

```
### Shapiro-Wilk Normality Test
shapiro.test( tolresids )

##
##  Shapiro-Wilk normality test
##
## data:  tolresids
## W = 0.9789, p-value = 0.8626

### Lilliefors test of normality.
### First install the 'nortest' R package
require( "nortest" )
lillie.test( tolresids )

##
##  Lilliefors (Kolmogorov-Smirnov) normality test
##
## data:  tolresids
## D = 0.09572, p-value = 0.8027
```

For both tests, the p-value exceeds a conventional α level, so we do not reject normality. This does not prove normality. Because fitted residuals are correlated, these tests on residuals are exploratory checks rather than exact tests of independent normal errors; also inspect the Q-Q plot.

3.3.4 Transformations

A possible remedial measure when the simple linear regression model is not appropriate is to transform the response and/or predictor variable. To transform a variable, we use the `dplyr::mutate( )` which comes from the `dplyr`

package. The `dplyr` package is installed as a dependency of `mosaic`. We call `dplyr::mutate()` explicitly below. The `dplyr::mutate()` function requires the name of the dataframe and the name of a new variable/object that will hold your transformed variable:

```
dplyr::mutate( 'dataframe name', 'new variable'= 'transformed variable')
```

To illustrate transformation, we use the data from Toluca Company example. Specifically, we want to apply a square root transformation to the response variable and a natural log transformation to the explanatory variable:

```
### Recall toluca.data
pillar::glimpse( toluca.data )
```

```
## Rows: 25
## Columns: 2
## $ xvar <int> 80, 30, 50, 90, 70, 60, 120, 80, 100, 50, 40, 70, 90, 20, 110, 10~
## $ yvar <int> 399, 121, 221, 376, 361, 224, 546, 352, 353, 157, 160, 252, 389, ~
```

```
### Apply the square root trans. to the
### response. This function returns a new dataframe
### with the added variable
dplyr::mutate( toluca.data, sqrt_y= sqrt( yvar ) ) # the square-root variable will be
```

```
##      xvar yvar      sqrt_y
## 1     80 399 19.97498
## 2     30 121 11.00000
## 3     50 221 14.86607
## 4     90 376 19.39072
## 5     70 361 19.00000
## 6     60 224 14.96663
## 7    120 546 23.36664
## 8     80 352 18.76166
## 9    100 353 18.78829
## 10    50 157 12.52996
## 11    40 160 12.64911
## 12    70 252 15.87451
## 13    90 389 19.72308
## 14    20 113 10.63015
## 15   110 435 20.85665
## 16   100 420 20.49390
## 17    30 212 14.56022
## 18    50 268 16.37071
## 19    90 377 19.41649
## 20   110 421 20.51828
```

```
## 21    30   273 16.52271
## 22    90   468 21.63331
## 23    40   244 15.62050
## 24    80   342 18.49324
## 25    70   323 17.97220
```

We want to be able to use the transformed variable, so we store the resulting new dataframe to a new object:

```
toluca.dataVer1 <- dplyr::mutate( toluca.data,  sqrty= sqrt( yvar ) )
pillar::glimpse( toluca.dataVer1 )
```

```
## Rows: 25
## Columns: 3
## $ xvar <int> 80, 30, 50, 90, 70, 60, 120, 80, 100, 50, 40, 70, 90, 20, 110, 1~
## $ yvar <int> 399, 121, 221, 376, 361, 224, 546, 352, 353, 157, 160, 252, 389,~
## $ sqrty <dbl> 19.97498, 11.00000, 14.86607, 19.39072, 19.00000, 14.96663, 23.3~
```

Now apply the natural log transformation to the explanatory variable:

```
### Apply natural log trans. to x.
### Note that we are using toluca.dataVer1
### dataframe since it already includes 'sqrty'.
toluca.dataVer2 <- dplyr::mutate( toluca.dataVer1,  lnx = log( xvar ) )
pillar::glimpse( toluca.dataVer2 )
```

```
## Rows: 25
## Columns: 4
## $ xvar <int> 80, 30, 50, 90, 70, 60, 120, 80, 100, 50, 40, 70, 90, 20, 110, 1~
## $ yvar <int> 399, 121, 221, 376, 361, 224, 546, 352, 353, 157, 160, 252, 389,~
## $ sqrty <dbl> 19.97498, 11.00000, 14.86607, 19.39072, 19.00000, 14.96663, 23.3~
## $ lnx <dbl> 4.382027, 3.401197, 3.912023, 4.499810, 4.248495, 4.094345, 4.78~
```

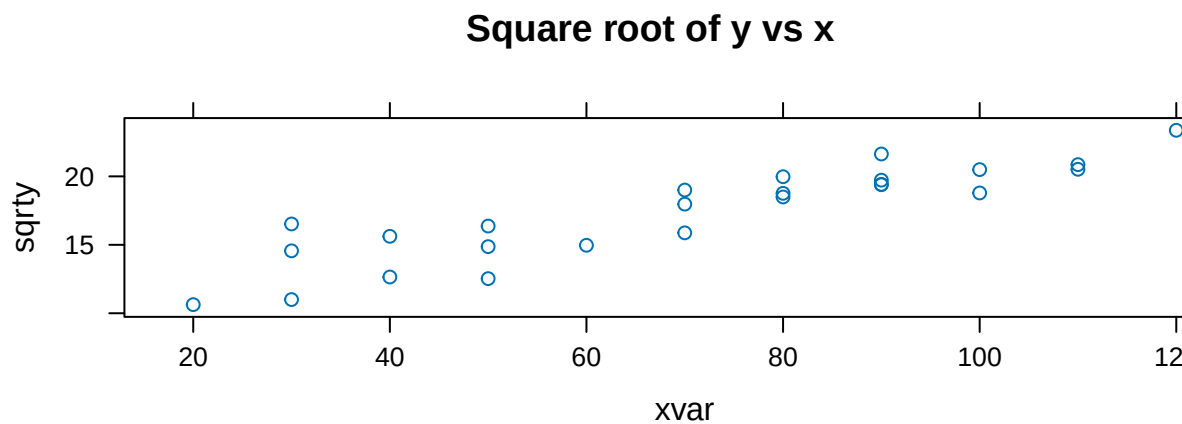
```
### One could also apply as many trans. as one would
### like by using mutate only once:
toluca.dataWithTrans <- dplyr::mutate( toluca.data,  sqrty= sqrt( yvar ), lnx = log( xvar ),
                                       lny = log( yvar ) , sqrd = yvar^2, cubertx = xvar^( 1/3 ) )
pillar::glimpse( toluca.dataWithTrans )
```

```
## Rows: 25
## Columns: 7
## $ xvar <int> 80, 30, 50, 90, 70, 60, 120, 80, 100, 50, 40, 70, 90, 20, 110,~
## $ yvar <int> 399, 121, 221, 376, 361, 224, 546, 352, 353, 157, 160, 252, 38~
```

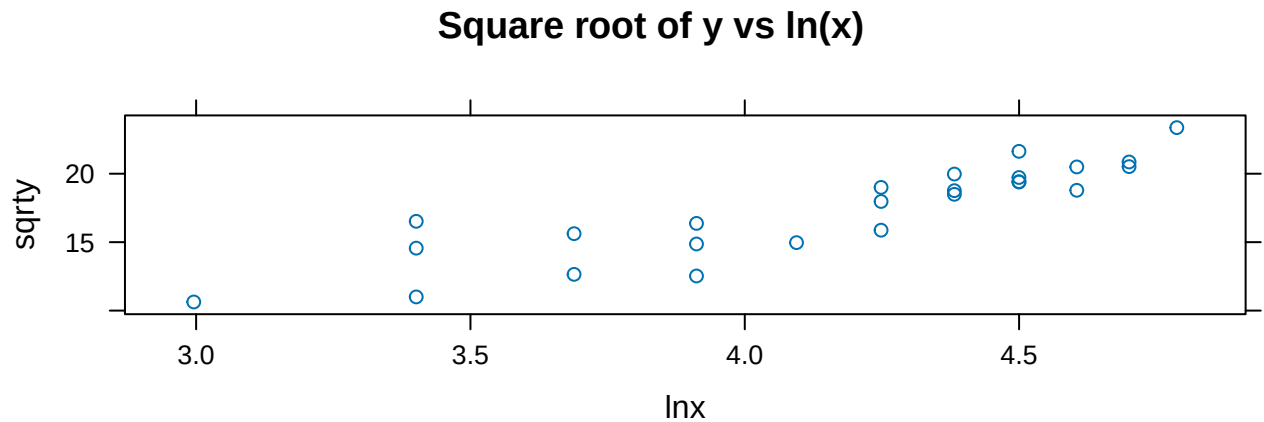
```
## $ sqrty    <dbl> 19.97498, 11.00000, 14.86607, 19.39072, 19.00000, 14.96663, 23~
## $ lnx      <dbl> 4.382027, 3.401197, 3.912023, 4.499810, 4.248495, 4.094345, 4.~
## $ lny      <dbl> 5.988961, 4.795791, 5.398163, 5.929589, 5.888878, 5.411646, 6.~
## $ sqrd     <dbl> 159201, 14641, 48841, 141376, 130321, 50176, 298116, 123904, 1~
## $ cubertx  <dbl> 4.308869, 3.107233, 3.684031, 4.481405, 4.121285, 3.914868, 4.~
```

Once the transformed variables are stored in a dataframe, you can plot them or fit the simple linear regression model:

```
# Note the dataframe name
xyplot(sqrty ~ xvar, data= toluca.dataWithTrans, main="Square root of y vs x" )
```



```
xyplot(sqrty ~ lnx, data= toluca.dataWithTrans, main="Square root of y vs ln(x)" )
```



```
toluca.fitA <- lm( sqrty ~ xvar , data= toluca.dataWithTrans )
toluca.fitA
```

```
##
## Call:
## lm(formula = sqrty ~ xvar, data = toluca.dataWithTrans)
##
## Coefficients:
## (Intercept)      xvar
##      9.9722      0.1055
```

3.3.5 F Test for Lack of Fit

The lack-of-fit test requires repeated observations at one or more x levels. To proceed, one may compute the MSLF and MSPE using the summation formulas using **R**. An easier approach is to determine the full and reduced model when testing for a lack of fit. Recall that in this setting we have the following models:

Full Model: $Y_{ij} = \mu_j + \varepsilon_{ij}$

Reduced Model: $Y_{ij} = \beta_0 + \beta_1 X_j + \varepsilon_{ij}$

If the full and reduced model can be determined (and are nested), then one may use the general F-test to test whether we can reject the null hypothesis (which postulates the reduced model). In **R**, the general F-test can be carried out using `anova( reduced model , full model , test= "F" )`. Note that in this case, the full model is a single-factor (potentially unbalanced) ANOVA model, while the reduced model is the straight-line regression. We follow the Bank Example from ALRM to illustrate this in **R**:

```
### Import data:
url <- "http://www.csub.edu/~emontoya2/datasets/textbookdata/Kutner/Chapter%20%203%20D"

bank.data <- read.csv( url , header = FALSE , col.names = c("x", "y") , sep = " " )

pillar::glimpse( bank.data )
```

```
## Rows: 11
## Columns: 2
## $ x <int> 125, 100, 200, 75, 150, 175, 75, 175, 125, 200, 100
## $ y <int> 160, 112, 124, 28, 152, 156, 42, 124, 150, 104, 136
```

```
### Fit full model, but first convert
### the explanatory into a factor variable.
### 'as.factor()' converts a variable into a factor variable.
bank.dataUD <- dplyr::mutate( bank.data, xfac = as.factor( x ) )

# levels() returns the level labels of a factor.
levels.x <- levels( bank.dataUD$xfac ) # determine the levels of X
levels.x
```

```
## [1] "75" "100" "125" "150" "175" "200"
```

```
n.bank <- nrow(bank.dataUD)
n.levels <- length(levels.x)
n.levels
```

```
## [1] 6
```

```
n.bank - n.levels
```

```
## [1] 5
```

```
### Full model:
### (Note: for those that have taken 4220: this is a one-way unbalanced aov problem)
fit.bank.full <- lm(y~ xfac , data= bank.dataUD)

### Reduced model:
fit.bank.reduced <- lm(y~ x , data= bank.dataUD)

### anova(reduced model, full model, test="F")
anova( fit.bank.reduced , fit.bank.full , test= "F" )
```

```
## Analysis of Variance Table
##
## Model 1: y ~ x
## Model 2: y ~ xfac
##   Res.Df  RSS Df Sum of Sq    F    Pr(>F)
## 1      9 14742
## 2      5 1148 4      13594 14.801 0.005594 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

What information is given by `anova( fit.bank.reduced , fit.bank.full ,`

AOV Table							
Model 1:	Y~X						
Model 2:	Y~factor.X						
	Res.Df	RSS	Df	Sum of Sq	F		Pr(> F)
1	n-2	SSE					
1	n-c	SSPE	c-2	SSLF	MSLF/MSPE		$P(F_{c-2, n-c} > F^*)$

`test="F")`? We obtain

3.4 Least absolute deviation (LAD) regression

Least squares minimizes the sum of squared residuals, $\sum_i e_i^2$. Least absolute deviation (LAD) regression minimizes the sum of absolute residuals, $\sum_i |e_i|$. Neither criterion requires normal errors just to compute estimates; distributional assumptions matter for inference.

LAD is less sensitive than least squares to large response residuals, but it can still be affected by unusual predictor values (high-leverage cases). Residual diagnostics remain useful when considering either method.

3.5 The matrix approach to simple linear regression

3.5.1 Creating a matrix

Recall that the function `c( )` creates a vector. For example,

```
xvec <- c( 4, 5, 6, 7 )
```

We can select a given element from this vector by placing `[ ]` after the vector name with a specified index:

```
### 1st element in the vector
xvec[1]
```

```
## [1] 4
```

```
xvec[3] # 3rd element in the vector
```

```
## [1] 6
```

A matrix is a rectangular way of storing data. You can think of it as simply a way to store data. Matrices will have two dimensions: rows and columns. That is, each matrix will consist of rows and columns of elements (or data).

The function `matrix()` allows a user to create a matrix from a vector of data. Four useful arguments are: `data` (a vector of data), `nrow` (desired number of rows), `ncol` (desired number of columns), and `byrow` (set equal to `FALSE` if the matrix is to be filled by column, set to `TRUE` if to be filled by row). The code below creates a vector of data to be transformed into a matrix:

```
a <- c(1,2,3,4,5,6)
```

```
A <- matrix( data= a, nrow= 2, ncol= 3, byrow= FALSE )
A
```

```
##      [,1] [,2] [,3]
## [1,]    1    3    5
## [2,]    2    4    6
```

```
B <- matrix( data= a, nrow= 2, ncol= 3, byrow= TRUE )
B
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    4    5    6
```

3.5.2 Some matrix operations

It may be necessary to perform certain matrix operations. In this section you will learn how to perform matrix multiplication, transpose a matrix, obtain the inverse of a matrix, and element by element multiplication. The commands that we will use are:

Name	What does it do?
%*%	Performs matrix multiplication
t()	Transposes a matrix.
solve()	Obtains an Inverse of a matrix
*	Performs element by element multiplication.

The commands are illustrated with the following code:

```
### Create two matrices
C= matrix( c( 1, 2, 1, -8, 26, 4, 5, 6, 10 ) , nrow= 3, ncol= 3 )
D= matrix( c( 7, 8, 9, 10, 11, 12, 13, 14, 15 ), nrow= 3, ncol= 3 )

C
```

```
##      [,1] [,2] [,3]
## [1,]    1   -8    5
## [2,]    2   26    6
## [3,]    1    4   10
```

```
D
```

```
##      [,1] [,2] [,3]
## [1,]    7   10   13
## [2,]    8   11   14
## [3,]    9   12   15
```

```
### Matrix multiplication
C%*%D      # multiply matrices C and D
```

```
##      [,1] [,2] [,3]
## [1,]  -12  -18  -24
## [2,]  276  378  480
## [3,]  129  174  219
```

```
### Matrix transpose
t(C)      # transpose of matrix C
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    1
## [2,]   -8   26    4
## [3,]    5    6   10
```

```
### element by element multiplication
C*D # this is NOT matrix multiplication
```

```
##      [,1] [,2] [,3]
## [1,]    7  -80   65
## [2,]   16  286   84
## [3,]    9   48  150
```

```
### obtain the inverse of C
solve(C)
```

```
##      [,1]      [,2]      [,3]
## [1,] 0.91472868 0.38759690 -0.68992248
## [2,] -0.05426357 0.01937984 0.01550388
## [3,] -0.06976744 -0.04651163 0.16279070
```

```
### Add two matrices
C + D
```

```
##      [,1] [,2] [,3]
## [1,]    8    2   18
## [2,]   10   37   20
## [3,]   10   16   25
```

3.5.3 LS estimation

The simple linear regression model in matrix form is

$$Y_{n \times 1} = X_{n \times 2} \beta_{2 \times 1} + \varepsilon_{n \times 1}$$

where

$$Y_{n \times 1} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad X_{n \times 2} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}, \quad \beta_{2 \times 1} = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}, \quad \text{and} \quad \varepsilon_{n \times 1} = \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}$$

The matrix approach is illustrated in **R** using the Toluca Company example:

```
### Recall the data
```

```
pillar::glimpse( toluca.data )
```

```
## Rows: 25
```

```
## Columns: 2
```

```
## $ xvar <int> 80, 30, 50, 90, 70, 60, 120, 80, 100, 50, 40, 70, 90, 20, 110, 10~
```

```
## $ yvar <int> 399, 121, 221, 376, 361, 224, 546, 352, 353, 157, 160, 252, 389, ~
```

```
### Extract both variables to set up the matrices
```

```
hrs <- toluca.data$yvar
```

```
lotsize <- toluca.data$xvar
```

```
### Set up the response column vector
```

```
yvec <- matrix( hrs, nrow= 25, ncol= 1)
```

```
yvec
```

```
##      [,1]
```

```
## [1,] 399
```

```
## [2,] 121
```

```
## [3,] 221
```

```
## [4,] 376
```

```
## [5,] 361
```

```
## [6,] 224
```

```
## [7,] 546
```

```
## [8,] 352
```

```
## [9,] 353
```

```
## [10,] 157
```

```
## [11,] 160
```

```
## [12,] 252
```

```
## [13,] 389
```

```
## [14,] 113
```

```
## [15,] 435
```

```
## [16,] 420
```

```
## [17,] 212
```

```
## [18,] 268
```

```
## [19,] 377
```

```
## [20,] 421
```

```
## [21,] 273
```

```
## [22,] 468
```

```
## [23,] 244
```

```
## [24,] 342
```

```
## [25,] 323
```

```
### Define the design/model matrix.
# First we need to define a vector of 1's:
ones <- rep( 1, times= 25)
ones
```

```
## [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

```
# If you have the columns of the matrix
# already defined, then we can create the
# matrix by binding the columns using 'cbind()'.
Xmat <- cbind( ones , lotsize )
Xmat
```

```
##      ones lotsize
## [1,]    1     80
## [2,]    1     30
## [3,]    1     50
## [4,]    1     90
## [5,]    1     70
## [6,]    1     60
## [7,]    1    120
## [8,]    1     80
## [9,]    1    100
## [10,]   1     50
## [11,]   1     40
## [12,]   1     70
## [13,]   1     90
## [14,]   1     20
## [15,]   1    110
## [16,]   1    100
## [17,]   1     30
## [18,]   1     50
## [19,]   1     90
## [20,]   1    110
## [21,]   1     30
## [22,]   1     90
## [23,]   1     40
## [24,]   1     80
## [25,]   1     70
```

The LS estimates may be obtained via $b = (X^T X)^{-1} X^T y$. In **R** we have:

```
XmatTxmat <- t(Xmat) %*% Xmat

solve( XmatTxmat ) %*% t(Xmat) %*% yvec

##           [,1]
## ones      62.365859
## lotsize    3.570202
```

These are the same estimates provided by `summary( )`.

The fitted values, $\hat{y} = Xb$, are obtained via:

```
### Store the LS estimates
best <- solve( XmatTxmat ) %*% t(Xmat) %*% yvec

Xmat %*% best # compare to fitted(toluca.fit)

##           [,1]
## [1,] 347.9820
## [2,] 169.4719
## [3,] 240.8760
## [4,] 383.6840
## [5,] 312.2800
## [6,] 276.5780
## [7,] 490.7901
## [8,] 347.9820
## [9,] 419.3861
## [10,] 240.8760
## [11,] 205.1739
## [12,] 312.2800
## [13,] 383.6840
## [14,] 133.7699
## [15,] 455.0881
## [16,] 419.3861
## [17,] 169.4719
## [18,] 240.8760
## [19,] 383.6840
## [20,] 455.0881
## [21,] 169.4719
## [22,] 383.6840
## [23,] 205.1739
## [24,] 347.9820
## [25,] 312.2800
```

These values may also be obtained by using the hat matrix:

```
### Hat matrix
Hatmat <- Xmat %*% solve( XmatTxmat ) %*% t(Xmat)

Hatmat %*% yvec # same as above
```

```
##           [,1]
## [1,] 347.9820
## [2,] 169.4719
## [3,] 240.8760
## [4,] 383.6840
## [5,] 312.2800
## [6,] 276.5780
## [7,] 490.7901
## [8,] 347.9820
## [9,] 419.3861
## [10,] 240.8760
## [11,] 205.1739
## [12,] 312.2800
## [13,] 383.6840
## [14,] 133.7699
## [15,] 455.0881
## [16,] 419.3861
## [17,] 169.4719
## [18,] 240.8760
## [19,] 383.6840
## [20,] 455.0881
## [21,] 169.4719
## [22,] 383.6840
## [23,] 205.1739
## [24,] 347.9820
## [25,] 312.2800
```

3.6 Multiple linear regression (MLR)

This section deals with the multiple regression model

$$Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i$$

where the β 's are the parameters, x 's are known constants, and we assume that $\varepsilon_i \stackrel{iid}{\sim} N(0, \sigma^2)$. To illustrate the model fitting with **R** we follow the Dwaine Studios Example from ALRM. Note that the model can be expressed in matrix form and any subsequent analyses can be done in terms of matrices. Matrices will be used to obtain the least-squares estimates as an illustration of using matrices in **R**, but the rest of the analyses will be done using `lm( )` and `anova( )`.

3.6.1 Least-Squares Estimates

Begin by importing the data:

```
### Import data:
url <- "http://www.csub.edu/~emontoya2/datasets/textbookdata/Kutner/Chapter%20%206%20Data%20Sets/"

dwaine.data <- read.csv( url , header = FALSE , col.names = c("x1", "x2", "y") , sep = " " )
pillar::glimpse( dwaine.data )

## Rows: 21
## Columns: 3
## $ x1 <dbl> 68.5, 45.2, 91.3, 47.8, 46.9, 66.1, 49.5, 52.0, 48.9, 38.4, 87.9, 7~
## $ x2 <dbl> 16.7, 16.8, 18.2, 16.3, 17.3, 18.2, 15.9, 17.2, 16.6, 16.0, 18.3, 1~
## $ y <dbl> 174.4, 164.4, 244.2, 154.6, 181.6, 207.5, 152.8, 163.2, 145.4, 137.~
```

A matrix approach can be used as follows:

```
n <- 21

yresp <- dwaine.data$y

Xmat <- cbind(rep(1, n), dwaine.data$x1, dwaine.data$x2) # design/model matrix

yrespvec <- matrix( yresp, ncol = 1, nrow=21)

b <- solve( t( Xmat )%*( Xmat ) )%*t( Xmat )%*yrespvec
b

##           [,1]
## [1,] -68.85707
## [2,]  1.45456
## [3,]  9.36550
```

We can also use `lm( )` to fit a MLR model. It is used in the same manner as before but we add additional explanatory variables by adding a `+` after each explanatory variable:

```
### Note the argument of lm():
### Response variable Explanatory variable + Explanatory variable + ...
dwaine.fit <- lm( y ~ x1 + x2 , data= dwaine.data)
dwaine.fit
```

```
##
## Call:
## lm(formula = y ~ x1 + x2, data = dwaine.data)
##
## Coefficients:
## (Intercept)          x1          x2
##    -68.857      1.455      9.366
```

3.6.2 Fitted values, ANOVA table, residuals

All previously discussed functions that extract information from an *lm* object such as the LS estimates, fitted values, ANOVA table, etc. can be used in the same manner when fitting a MLR model. Recall that `coef( )` returns the LS estimates:

```
coef(dwaine.fit)
```

```
## (Intercept)          x1          x2
##    -68.85707      1.45456      9.36550
```

The fitted values and residuals can be obtained via:

```
fitted(dwaine.fit)
```

```
##          1          2          3          4          5          6          7          8
## 187.1841 154.2294 234.3963 153.3285 161.3849 197.7414 152.0551 167.8666
##          9         10         11         12         13         14         15         16
## 157.7382 136.8460 230.3874 197.1849 222.6857 141.5184 174.2132 228.1239
##         17         18         19         20         21
## 145.7470 159.0013 230.9870 230.3161 157.0644
```

```
residuals( dwaine.fit )
```

```
##          1          2          3          4          5          6
## -12.7841146 10.1705737  9.8036764  1.2714690 20.2150722  9.7585779
##          7          8          9         10         11         12
##  0.7449178 -4.6666316 -12.3381967  0.3539791 11.5126289 -6.0849209
##         13         14         15         16         17         18
##  9.3142995  3.7815611 -13.1132116 -18.4238900  0.6530062 -15.0013134
##         19         20         21
##  1.6129777 -6.2160615  9.4356009
```

To obtain the ANOVA table, run the following:

```
anova( dwaine.fit )
```

```
## Analysis of Variance Table
##
## Response: y
##           Df Sum Sq Mean Sq F value    Pr(>F)
## x1          1 23371.8  23371.8 192.8962 4.64e-11 ***
## x2          1   643.5    643.5   5.3108 0.03332 *
## Residuals 18   2180.9    121.2
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

What information is given by `anova( dwaine.fit )`? We obtain

AOV Table Response: Y					
	Df	Sum Sq	Mean Sq	F value	Pr(> F)
X1	1	$SSR(X_1)$	$MSR(X_1)=SSR(X_1)/1 = \diamond$	$F_1^* = \diamond/\heartsuit$	$P(F_{1,n-3} > F_1^*)$
X2	1	$SSR(X_2 X_1)$	$MSR(X_2 X_1)=SSR(X_2 X_1)/1 = \clubsuit$	$F_2^* = \clubsuit/\heartsuit$	$P(F_{1,n-3} > F_2^*)$
Residuals	$n - p$	$SSE(X_1, X_2)$	$MSE(X_1, X_2)=SSE(X_1, X_2)/n - 3 = \heartsuit$		

The ANOVA table returned by **R** gives the decomposition of $SSR(X_1, X_2)$ (Chapter 7 material). Note that $SSR(X_1, X_2) = SSR(X_1) + SSR(X_2|X_1)$.

3.6.3 Inference about the regression parameters

To determine the significance of the regression parameter(s) we can summarize the fit:

```
summary( dwaine.fit )
```

```
##
## Call:
## lm(formula = y ~ x1 + x2, data = dwaine.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -18.4239  -6.2161   0.7449   9.4356  20.2151
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -68.8571    60.0170  -1.147  0.2663
## x1           1.4546     0.2118   6.868 2e-06 ***
## x2           9.3655     4.0640   2.305 0.0333 *
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 11.01 on 18 degrees of freedom
## Multiple R-squared:  0.9167, Adjusted R-squared:  0.9075
## F-statistic: 99.1 on 2 and 18 DF, p-value: 1.921e-10
```

Note that the following is printed by `summary(dwaine.fit)`:

Coefficients:				
	Estimate	Std. Error	t value	$Pr(> t)$
(Intercept)	b_0	$s\{b_0\}$	$t_{b_0}^* = \frac{b_0}{s\{b_0\}}$	$2P(t_{n-2} > t_{b_0}^*)$
X1	b_1	$s\{b_1\}$	$t_{b_1}^* = \frac{b_1}{s\{b_1\}}$	$2P(t_{n-2} > t_{b_1}^*)$
X2	b_2	$s\{b_2\}$	$t_{b_2}^* = \frac{b_2}{s\{b_2\}}$	$2P(t_{n-2} > t_{b_2}^*)$

Residual standard error: $\sqrt{MSE(X_1, X_2)}$ on $n - p$ degrees of freedom

Multiple R-Squared: R^2 , Adjusted R-squared: $R_{adjusted}^2$

F-statistic: $F^* = \frac{SSR(X_1, X_2)/(p-1)}{SSE(X_1, X_2)/(n-p)} = \frac{MSR(X_1, X_2)}{MSE(X_1, X_2)}$ on $p - 1$ and $n - p$ DF, p-value: $P(F_{p-1, n-p} > F^*)$

Note that we are provided with the test statistic F^* to test whether sales are related to target population and per capita disposable income ($\beta_1 = 0$ and $\beta_2 = 0$), and we are provided with the test statistic t^* for each regression parameter.

3.6.4 Inference about the Mean Response and Prediction of New Observation

As before, we can use `predict()` to obtain a CI for the mean response and a PI for a predicted response. For example, a 95% confidence interval for $E(y_h)$ (expected sales in cities) in a community with $x_{h1} = 65.4$ (number of persons aged 16 or younger) and $x_{h2} = 17.6$ (per capita disposable personal income) is obtained in **R** by:

```
new.x.data <- data.frame(x1=65.4, x2=17.6)

predict(dwaine.fit, newdata=new.x.data, interval="confidence", level=.95)

##          fit          lwr          upr
## 1 191.1039 185.2911 196.9168
```

A 95% PI at these same levels of x_1 and x_2 is:

```
predict(dwaine.fit, newdata=new.x.data, interval="prediction", level=.95)
```

```
##          fit      lwr      upr
## 1 191.1039 167.2589 214.949
```

To predict at two different communities in which in one community the number of persons aged 16 or younger is 65.4 and the per capita disposable personal income is 17.6, and the other has the number of persons aged 16 or younger is 53.1 and the per capita disposable personal income is 17.7 we enter the following:

```
new.x.data2 <- data.frame(x1=c(65.4, 53.1), x2=c(17.6, 17.7))

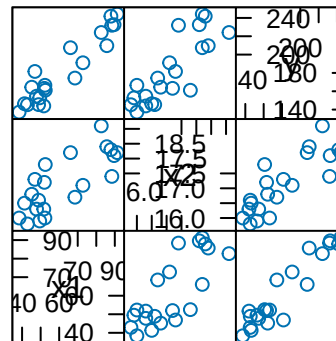
predict.lm(dwaine.fit, newdata=new.x.data2, interval="prediction", level=.95)
```

```
##          fit      lwr      upr
## 1 191.1039 167.2589 214.9490
## 2 174.1494 149.0867 199.2121
```

3.6.5 Diagnostics

Diagnostics are conducted in the same manner (QQ-plot of the residuals, scatterplots of the residuals versus $\hat{y}$, scatterplots of the residuals versus $\hat{y}$ versus a given x) as when the simple linear regression model was discussed, but now we also consider the scatter plot matrix and correlation matrix. The argument for both of these functions will be a dataframe.

```
splom( dwaine.data) # scatterplot matrix
```



Scatter Plot Matrix

```
cor( dwaine.data ) # correlation matrix
```

```
##           x1           x2           y
## x1 1.0000000 0.7812993 0.9445543
## x2 0.7812993 1.0000000 0.8358025
## y  0.9445543 0.8358025 1.0000000
```

3.6.6 ANOVA table containing decomposition of SSR

Here **R** is used to obtain an anova table with a decomposition of the SSR. We follow the Body Fat example from ALRM using **R** to obtain the anova table with the decomposition of SSR. The `anova( )` function will automatically decompose the SSR with the decomposition depending on the order that you enter the predictor variables into `lm( )`. For example, to get the decomposition

$$SSR(x_1, x_2, x_3) = SSR(x_3) + SSR(x_2|x_3) + SSR(x_1|x_3, x_2)$$

we enter the predictors into `lm()` as `y ~ x3 + x2 + x1`.

```
### Import data:
url <- "https://www.csub.edu/~emontoya2/datasets/textbookdata/Kutner/Chapter%20%207%201"

### This is not a .csv file and the file does not
### include variable names, so some changes
### have to be made to read.csv().
bf.data <- read.csv( url , header = FALSE , col.names = c( "x1", "x2", "x3", "y" ) , s

pillar::glimpse( bf.data )
```

```
## Rows: 20
## Columns: 4
## $ x1 <dbl> 19.5, 24.7, 30.7, 29.8, 19.1, 25.6, 31.4, 27.9, 22.1, 25.5, 31.1, 3~
## $ x2 <dbl> 43.1, 49.8, 51.9, 54.3, 42.2, 53.9, 58.5, 52.1, 49.9, 53.5, 56.6, 5~
## $ x3 <dbl> 29.1, 28.2, 37.0, 31.1, 30.9, 23.7, 27.6, 30.6, 23.2, 24.8, 30.0, 2~
## $ y <dbl> 11.9, 22.8, 18.7, 20.1, 12.9, 21.7, 27.1, 25.4, 21.3, 19.3, 25.4, 2~
```

```
bf.fit=lm(y~ x3 + x2 + x1, data=bf.data)
```

```
anova(bf.fit)
```

```
## Analysis of Variance Table
##
## Response: y
```

```
##           Df Sum Sq Mean Sq F value    Pr(>F)
## x3          1  10.05   10.05  1.6343    0.2193
## x2          1 374.23  374.23 60.8471 7.684e-07 ***
## x1          1  12.70   12.70  2.0657    0.1699
## Residuals 16  98.40    6.15
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

One could also use the extra sums of squares formulas to get the decomposition desired.

3.6.7 Testing whether $\beta_k \neq 0$ for some k

There are several ways to go about testing whether a single or multiple regression coefficients are significant using **R**. For example, we could get the appropriate extra sums of squares for the reduced model and the full model to calculate the appropriate test statistic. If the reduced model is nested under the full model then we can use `anova( )` to compute the test statistic and corresponding p-value. All that is required is to specify the full model and the model under the null hypothesis (reduced model).

Suppose it is of interest to test the following hypothesis for the Body Fat example from ALRM:

$$H_0 : \beta_2 = \beta_3 = 0$$

The reduced model is $Y_i = \beta_0 + \beta_1 X_{i1} + \varepsilon_i$ and the full model is $y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} + \varepsilon_i$. Clearly the models are nested therefore we use the function `anova( )`:

```
reduced.fit <- lm( y ~ x1 , data= bf.data )

full.fit=lm( y ~ x1 + x2 + x3 , data=bf.data)

anova( reduced.fit , full.fit , test= "F" )

## Analysis of Variance Table
##
## Model 1: y ~ x1
## Model 2: y ~ x1 + x2 + x3
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      18 143.120
## 2      16  98.405  2   44.715 3.6352 0.04995 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The following is printed by `anova( reduced.fit , full.fit, test='F' )`:

AOV Table						
M1:	Y~X1					
M2:	Y~X1+X2+X3					
	Res.Df	RSS	Df	Sum of Sq	F	Pr(> F)
1	n-2	SSE(X_1)				
2	n-4	SSE(X_1, X_2, X_3)	-2+4	SSR($X_2, X_3 X_1$)	$\frac{MSR(X_2, X_3 X_1)}{MSE(X_1, X_2, X_3)}$	$P(F_{2, n-4} > F^*)$

3.7 Interactions

Suppose we wish to include an interaction between x_1 and x_2 . In an R model formula, `x1*x2` expands to the two main effects and their interaction: `x1 + x2 + x1:x2`. Alternatively, use `dplyr::mutate()` to add a column containing the product `x1*x2`, and include that column alongside the main effects. Both methods are illustrated below:

```
BodyFat.Inter=lm(y~ x1*x2 + x3, data=bf.data)
summary(BodyFat.Inter)
```

```
##
## Call:
## lm(formula = y ~ x1 * x2 + x3, data = bf.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.5365 -1.8682  0.2772  1.2493  4.1589
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 150.87925  124.06427   1.216   0.243
## x1           4.15688    3.11251   1.336   0.202
## x2          -3.64609    3.11348  -1.171   0.260
## x3          -2.45340    1.72707  -1.421   0.176
## x1:x2         0.01401    0.02911   0.481   0.637
##
## Residual standard error: 2.542 on 15 degrees of freedom
## Multiple R-squared:  0.8044, Adjusted R-squared:  0.7522
## F-statistic: 15.42 on 4 and 15 DF,  p-value: 3.41e-05
```

```
bf.data2 <- dplyr::mutate( bf.data, x1x2= x1*x2)
BodyFat.Inter2=lm(y~ x1 + x2 + x3 + x1x2, data=bf.data2 )
summary(BodyFat.Inter2)
```

```
##
```

```
## Call:
## lm(formula = y ~ x1 + x2 + x3 + x1x2, data = bf.data2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.5365 -1.8682  0.2772  1.2493  4.1589
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 150.87925  124.06427   1.216   0.243
## x1           4.15688    3.11251   1.336   0.202
## x2          -3.64609    3.11348  -1.171   0.260
## x3          -2.45340    1.72707  -1.421   0.176
## x1x2          0.01401    0.02911   0.481   0.637
##
## Residual standard error: 2.542 on 15 degrees of freedom
## Multiple R-squared:  0.8044, Adjusted R-squared:  0.7522
## F-statistic: 15.42 on 4 and 15 DF,  p-value: 3.41e-05
```

3.8 Standardized multiple regression

To compare the size of the estimated regression parameter estimates on a common scale, we have to standardize each variable (response and explanatory) in the model. Standardized multiple regression in **R** is illustrated using the Dwaine studios data:

```
require( mosaic )
### Import data:
url <- "http://www.csub.edu/~emontoya2/datasets/textbookdata/Kutner/Chapter%20%206%20Data%20Sets/"

dwaine.data <- read.csv( url , header = FALSE , col.names = c("x1", "x2", "y") , sep = " " )
pillar::glimpse( dwaine.data )
```

```
## Rows: 21
## Columns: 3
## $ x1 <dbl> 68.5, 45.2, 91.3, 47.8, 46.9, 66.1, 49.5, 52.0, 48.9, 38.4, 87.9, 7~
## $ x2 <dbl> 16.7, 16.8, 18.2, 16.3, 17.3, 18.2, 15.9, 17.2, 16.6, 16.0, 18.3, 1~
## $ y <dbl> 174.4, 164.4, 244.2, 154.6, 181.6, 207.5, 152.8, 163.2, 145.4, 137.~
```

To standardize a given variable, we use a function called `scale( )` within `mutate`:

```

require( mosaic )
### Standardize each explanatory variable using dplyr::mutate()
### with scale().
dwaine.dataStdz <- dplyr::mutate( dwaine.data , y.stdz = as.numeric(scale( y )) ,
                                x1.stdz = as.numeric(scale( x1 )) ,
                                x2.stdz = as.numeric(scale( x2 )) )

# The head( ) function allows one to view the
# first few observations of a given dataset.
# Note that we now have each standardized
# variable.
head( dwaine.dataStdz )

##      x1  x2    y    y.stdz  x1.stdz  x2.stdz
## 1 68.5 16.7 174.4 -0.20736368  0.3480579 -0.4563909
## 2 45.2 16.8 164.4 -0.48367315 -0.9032627 -0.3533349
## 3 91.3 18.2 244.2  1.72127642  1.5725261  1.0894493
## 4 47.8 16.3 154.6 -0.75445643 -0.7636303 -0.8686150
## 5 46.9 17.3 181.6 -0.00842086 -0.8119646  0.1619452
## 6 66.1 18.2 207.5  0.70722067  0.2191665  1.0894493

dwaine.fitStdz <- lm( y.stdz ~ x1.stdz + x2.stdz, data= dwaine.dataStdz)
summary(dwaine.fitStdz)

##
## Call:
## lm(formula = y.stdz ~ x1.stdz + x2.stdz, data = dwaine.dataStdz)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.50907 -0.17176  0.02058  0.26071  0.55856
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -5.120e-17  6.637e-02   0.000   1.0000
## x1.stdz      7.484e-01  1.090e-01   6.868   2e-06 ***
## x2.stdz      2.511e-01  1.090e-01   2.305   0.0333 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3041 on 18 degrees of freedom
## Multiple R-squared:  0.9167, Adjusted R-squared:  0.9075
## F-statistic: 99.1 on 2 and 18 DF, p-value: 1.921e-10

```

Standardization changes the coefficient scales and residual standard error, but preserves fitted relationships, R^2 , and the slope tests. Retaining the intercept preserves the usual centered R^2 definition; the fitted intercept is approximately zero.

3.9 Polynomial regression

In this section, we start by using R to fit the second-order polynomial regression model with one predictor variable:

$$y_i = \beta_0 + \beta_1 x_i^* + \beta_2 x_i^{*2} + \varepsilon_i$$

where $x_i^* = x_i - \bar{x}$.

We fit this polynomial model using New York air quality measurements provided in the `airquality` dataset. In this illustration, the response is ozone and the explanatory variable is temperature. To start, center the explanatory variable and square the centered explanatory variable:

```
require( mosaic )

data( airquality )
pillar::glimpse( airquality )

## Rows: 153
## Columns: 6
## $ Ozone   <int> 41, 36, 12, 18, NA, 28, 23, 19, 8, NA, 7, 16, 11, 14, 18, 14, ~
## $ Solar.R <int> 190, 118, 149, 313, NA, NA, 299, 99, 19, 194, NA, 256, 290, 27~
## $ Wind    <dbl> 7.4, 8.0, 12.6, 11.5, 14.3, 14.9, 8.6, 13.8, 20.1, 8.6, 6.9, 9~
## $ Temp    <int> 67, 72, 74, 62, 56, 66, 65, 59, 61, 69, 74, 69, 66, 68, 58, 64~
## $ Month   <int> 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, ~
## $ Day     <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, ~

# center the explanatory variable and square the
# centered explanatory variable
airqualityCen <- dplyr::mutate( airquality, cenx1 = as.numeric(scale( Temp, scale = FALSE )), cen
```

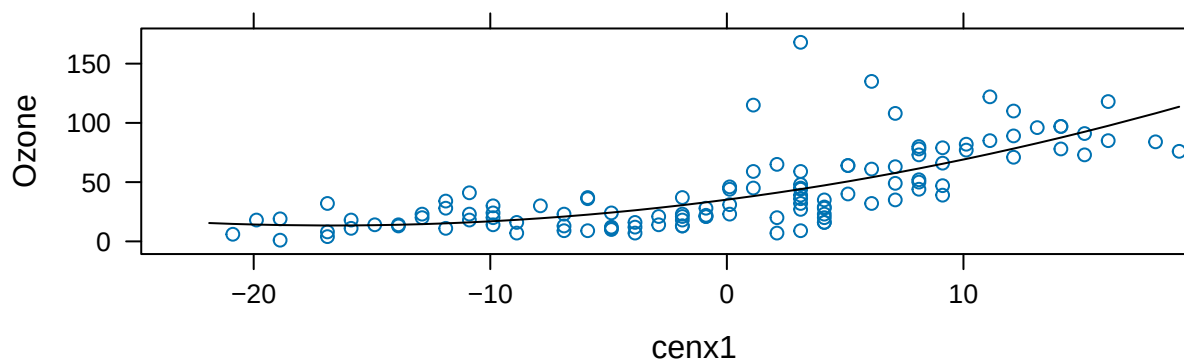
A polynomial regression is a linear regression model, so we fit the model in the same manner as past models:

```
require( mosaic )
polyfit <- lm( Ozone~ cenx1 + cenx1sq , data= airqualityCen )

# summarize the fit
summary( polyfit )
```

```
##
## Call:
## lm(formula = Ozone ~ cenx1 + cenx1sq, data = airqualityCen)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -37.619 -12.513  -2.736   9.676 123.909
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  35.19617    2.79472  12.594 < 2e-16 ***
## cenx1         2.60963    0.22610   11.542 < 2e-16 ***
## cenx1sq       0.07807    0.02086    3.743 0.000288 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 22.47 on 113 degrees of freedom
## (37 observations deleted due to missingness)
## Multiple R-squared:  0.5442, Adjusted R-squared:  0.5362
## F-statistic: 67.46 on 2 and 113 DF,  p-value: < 2.2e-16
```

```
xyplot( Ozone ~ cenx1 , data = airqualityCen, panel = function(x, y, ...){
  panel.xyplot(x, y, ...)
  fm <- lm(y ~ poly(x, 2))
  panel.lines(sort(x), predict(fm, newdata=data.frame( x=sort(x))), col.line = "black", lty=1)
} )
```



3.10 Model selection

This section deals with different criteria for model selection for MLR. To illustrate different model selection criteria with R, we follow the Surgical Unit Example from ALRM. To start, import the data:

```
url <- "http://www.csub.edu/~emontoya2/datasets/textbookdata/Kutner/Chapter%20%209%20Data%20Sets/"

surgunit.data <- read.csv( url , header = FALSE , col.names = c("x1", "x2", "x3", "x4", "x5", "x6", "x7", "x8", "y", "lny") )
pillar::glimpse( surgunit.data )

## Rows: 54
## Columns: 10
## $ x1  <dbl> 6.7, 5.1, 7.4, 6.5, 7.8, 5.8, 5.7, 3.7, 6.0, 3.7, 6.3, 6.7, 5.8, 5.~
## $ x2  <int> 62, 59, 57, 73, 65, 38, 46, 68, 67, 76, 84, 51, 96, 83, 62, 74, 85~
## $ x3  <int> 81, 66, 83, 41, 115, 72, 63, 81, 93, 94, 83, 43, 114, 88, 67, 68, ~
## $ x4  <dbl> 2.59, 1.70, 2.16, 2.01, 4.30, 1.42, 1.91, 2.57, 2.50, 2.40, 4.13, ~
## $ x5  <int> 50, 39, 55, 48, 45, 65, 49, 69, 58, 48, 37, 57, 63, 52, 58, 64, 36~
## $ x6  <int> 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, ~
## $ x7  <int> 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, ~
## $ x8  <int> 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, ~
## $ y    <int> 695, 403, 710, 349, 2343, 348, 518, 749, 1056, 968, 745, 257, 1573~
## $ lny  <dbl> 6.544, 5.999, 6.565, 5.854, 7.759, 5.852, 6.250, 6.619, 6.962, 6.8~
```

Tell R that certain variables are categorical:

```
surgunit.data <- dplyr::mutate( surgunit.data, x6fac =as.factor(x6), x7fac = as.factor( x7 ), x8fac = as.factor(x8) )
pillar::glimpse( surgunit.data )

## Rows: 54
## Columns: 13
## $ x1  <dbl> 6.7, 5.1, 7.4, 6.5, 7.8, 5.8, 5.7, 3.7, 6.0, 3.7, 6.3, 6.7, 5.8, ~
## $ x2  <int> 62, 59, 57, 73, 65, 38, 46, 68, 67, 76, 84, 51, 96, 83, 62, 74, ~
## $ x3  <int> 81, 66, 83, 41, 115, 72, 63, 81, 93, 94, 83, 43, 114, 88, 67, 68~
## $ x4  <dbl> 2.59, 1.70, 2.16, 2.01, 4.30, 1.42, 1.91, 2.57, 2.50, 2.40, 4.13~
## $ x5  <int> 50, 39, 55, 48, 45, 65, 49, 69, 58, 48, 37, 57, 63, 52, 58, 64, ~
## $ x6  <int> 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, ~
## $ x7  <int> 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1~
## $ x8  <int> 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0~
## $ y    <int> 695, 403, 710, 349, 2343, 348, 518, 749, 1056, 968, 745, 257, 15~
## $ lny  <dbl> 6.544, 5.999, 6.565, 5.854, 7.759, 5.852, 6.250, 6.619, 6.962, 6~
## $ x6fac <fct> 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1~
## $ x7fac <fct> 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1~
## $ x8fac <fct> 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0~
```

3.10.1 $R_{a,p}^2$ and C_p

For nested least-squares models fitted to the same observations with an intercept, R^2 cannot decrease when predictors are added. The adjusted coefficient of determination, $R_{a,p}^2$, may decrease as the number of covariates in the model increases, and so takes into account the number of explanatory variables that are in the model. To compute $R_{a,p}^2$ for several candidate models, we use the `leaps()` function from the `leaps` R package. Install `leaps` once with `install.packages("leaps")`, then load it in each session:

```
require( leaps )
```

First, we have to define a matrix whose columns consist of the variables under consideration (this is not the design/model matrix). `leaps()` expects numeric predictor columns, not factor objects. Keep the numeric codes for the binary variables x_6 , x_7 , and x_8 rather than using their factored versions. A categorical predictor with more than two levels would need multiple indicator columns.

```
Xleaps <- as.matrix(dplyr::select(surgunit.data, x1, x2, x3, x4, x5, x6, x7, x8))
```

The `leaps()` function requires this matrix, the response variable and the model selection criterion ("`Cp`" or "`adjr2`"):

```
R2adj.temp <- leaps(x=Xleaps, y=surgunit.data$`lry`, method="adjr2")
```

To better organize the information provided by `leaps()`, run the following:

```
#obtain relevant info
R2adj.info=cbind(R2adj.temp$which,R2adj.temp$size,R2adj.temp$adjr2)

# add names to make it easier to read info
colnames(R2adj.info)=c("x1","x2","x3","x4","x5", "x6", "x7", "x8", "p","adjr2")

R2adj.info
```

```
##      x1 x2 x3 x4 x5 x6 x7 x8 p      adjr2
## 1  0  0  1  0  0  0  0  0  2  0.416557881
## 1  0  0  0  1  0  0  0  0  2  0.410417798
## 1  0  1  0  0  0  0  0  0  2  0.205862939
## 1  0  0  0  0  0  0  0  1  2  0.122406556
## 1  1  0  0  0  0  0  0  0  2  0.042543244
## 1  0  0  0  0  0  1  0  0  2  0.035653717
## 1  0  0  0  0  1  0  0  0  2  0.002187716
## 1  0  0  0  0  0  0  1  0  2 -0.002898254
```

```
## 2 0 1 1 0 0 0 0 0 3 0.650085544
## 2 0 0 1 1 0 0 0 0 3 0.583777218
## 2 1 0 1 0 0 0 0 0 3 0.530934014
## 2 0 0 1 0 0 0 0 1 3 0.497419295
## 2 0 0 0 1 0 0 0 1 3 0.488519841
## 2 0 1 0 1 0 0 0 0 3 0.462691631
## 2 0 0 1 0 0 1 0 0 3 0.426095576
## 2 0 0 1 0 1 0 0 0 3 0.424491387
## 2 0 0 0 1 0 0 1 0 3 0.411731492
## 2 0 0 1 0 0 0 1 0 3 0.410390386
## 3 0 1 1 0 0 0 0 1 4 0.764715749
## 3 1 1 1 0 0 0 0 0 4 0.742729360
## 3 0 1 1 1 0 0 0 0 4 0.700885337
## 3 0 1 1 0 0 0 1 0 4 0.661818074
## 3 0 1 1 0 1 0 0 0 4 0.656711027
## 3 0 1 1 0 0 1 0 0 4 0.649910695
## 3 0 0 1 1 0 0 0 1 4 0.649267958
## 3 1 0 1 1 0 0 0 0 4 0.588850588
## 3 0 0 1 1 0 0 1 0 4 0.581680554
## 3 0 0 1 1 1 0 0 0 4 0.577651443
## 4 1 1 1 0 0 0 0 1 5 0.815996984
## 4 0 1 1 1 0 0 0 1 5 0.799293694
## 4 0 1 1 0 0 1 0 1 5 0.771518218
## 4 0 1 1 0 1 0 0 1 5 0.765894154
## 4 0 1 1 0 0 0 1 1 5 0.761994941
## 4 1 1 1 0 1 0 0 0 5 0.750009741
## 4 1 1 1 0 0 0 1 0 5 0.747435371
## 4 1 1 1 0 0 1 0 0 5 0.741872034
## 4 1 1 1 1 0 0 0 0 5 0.739554570
## 4 0 1 1 1 0 0 1 0 5 0.711129897
## 5 1 1 1 0 0 1 0 1 6 0.820508061
## 5 1 1 1 0 1 0 0 1 6 0.818704958
## 5 1 1 1 1 0 0 0 1 6 0.815758658
## 5 1 1 1 0 0 0 1 1 6 0.814141607
## 5 0 1 1 1 0 1 0 1 6 0.798911508
## 5 0 1 1 1 0 0 1 1 6 0.796775100
## 5 0 1 1 1 1 0 0 1 6 0.796460487
## 5 0 1 1 0 1 1 0 1 6 0.772967184
## 5 0 1 1 0 0 1 1 1 6 0.768818413
## 5 0 1 1 0 1 0 1 1 6 0.764075717
## 6 1 1 1 0 1 1 0 1 7 0.823449402
## 6 1 1 1 0 0 1 1 1 7 0.818660139
## 6 1 1 1 1 0 1 0 1 7 0.818132982
## 6 1 1 1 0 1 0 1 1 7 0.817824375
## 6 1 1 1 1 1 0 0 1 7 0.816356244
## 6 1 1 1 1 0 0 1 1 7 0.813704868
```

```
## 6 0 1 1 1 0 1 1 1 7 0.796331530
## 6 0 1 1 1 1 1 0 1 7 0.796311668
## 6 0 1 1 1 1 0 1 1 7 0.794252288
## 6 0 1 1 0 1 1 1 1 7 0.771189602
## 7 1 1 1 0 1 1 1 1 8 0.822597377
## 7 1 1 1 1 1 1 0 1 8 0.819816836
## 7 1 1 1 1 0 1 1 1 8 0.816097625
## 7 1 1 1 1 1 0 1 1 8 0.815143680
## 7 0 1 1 1 1 1 1 1 8 0.794098517
## 7 1 1 1 1 1 1 1 0 8 0.746632006
## 7 1 0 1 1 1 1 1 1 8 0.634657905
## 7 1 1 0 1 1 1 1 1 8 0.553658658
## 8 1 1 1 1 1 1 1 1 9 0.818773720
```

Note that models with higher $R^2_{a,p}$ are the models that include x_1 , x_2 , x_3 , and x_8 and include x_1 , x_2 , x_3 , x_6 , and x_8 .

Mallows' C_p criterion may also be obtained in the same manner using `leaps` but we replace "adjr2" with "Cp":

```
Cp.temp <- leaps(x=Xleaps, y=surgunit.data$lny, method="Cp")

#obtain relevant info
Cp.info=cbind(Cp.temp$which,Cp.temp$size,Cp.temp$Cp)

# add names to make it easier to read info
colnames(Cp.info)=c("x1","x2","x3","x4","x5", "x6", "x7", "x8", "p","Cp")

Cp.info
```

```
##   x1 x2 x3 x4 x5 x6 x7 x8 p      Cp
## 1  0  0  1  0  0  0  0  2 117.409441
## 1  0  0  0  1  0  0  0  2 119.171240
## 1  0  1  0  0  0  0  0  2 177.865004
## 1  0  0  0  0  0  0  1  2 201.811487
## 1  1  0  0  0  0  0  0  2 224.726995
## 1  0  0  0  0  0  1  0  2 226.703835
## 1  0  0  0  0  1  0  0  2 236.306372
## 1  0  0  0  0  0  0  1  2 237.765710
## 2  0  1  1  0  0  0  0  3  50.471575
## 2  0  0  1  1  0  0  0  3  69.131808
## 2  1  0  1  0  0  0  0  3  84.002738
## 2  0  0  1  0  0  0  1  3  93.434321
## 2  0  0  0  1  0  0  1  3  95.938771
## 2  0  1  0  1  0  0  0  3 103.207247
```

```

## 2 0 0 1 0 0 1 0 0 3 113.505967
## 2 0 0 1 0 1 0 0 0 3 113.957411
## 2 0 0 0 1 0 0 1 0 3 117.548252
## 2 0 0 1 0 0 0 1 0 3 117.925661
## 3 0 1 1 0 0 0 0 1 4 18.914496
## 3 1 1 1 0 0 0 0 0 4 24.980500
## 3 0 1 1 1 0 0 0 0 4 36.525190
## 3 0 1 1 0 0 0 1 0 4 47.303776
## 3 0 1 1 0 1 0 0 0 4 48.712801
## 3 0 1 1 0 0 1 0 0 4 50.589000
## 3 0 0 1 1 0 0 0 1 4 50.766330
## 3 1 0 1 1 0 0 0 0 4 67.435372
## 3 0 0 1 1 0 0 1 0 4 69.413572
## 3 0 0 1 1 1 0 0 0 4 70.525196
## 4 1 1 1 0 0 0 0 1 5 5.750774
## 4 0 1 1 1 0 0 0 1 5 10.267014
## 4 0 1 1 0 0 1 0 1 5 17.776952
## 4 0 1 1 0 1 0 0 1 5 19.297588
## 4 0 1 1 0 0 0 1 1 5 20.351858
## 4 1 1 1 0 1 0 0 0 5 23.592419
## 4 1 1 1 0 0 0 1 0 5 24.288478
## 4 1 1 1 0 0 1 0 0 5 25.792694
## 4 1 1 1 1 0 0 0 0 5 26.419290
## 4 0 1 1 1 0 0 1 0 5 34.104759
## 5 1 1 1 0 0 1 0 1 6 5.540639
## 5 1 1 1 0 1 0 0 1 6 6.018212
## 5 1 1 1 1 0 0 0 1 6 6.798576
## 5 1 1 1 0 0 0 1 1 6 7.226872
## 5 0 1 1 1 0 1 0 1 6 11.260750
## 5 0 1 1 1 0 0 1 1 6 11.826604
## 5 0 1 1 1 1 0 0 1 6 11.909933
## 5 0 1 1 0 1 1 0 1 6 18.132422
## 5 0 1 1 0 0 1 1 1 6 19.231275
## 5 0 1 1 0 1 0 1 1 6 20.487436
## 6 1 1 1 0 1 1 0 1 7 5.787389
## 6 1 1 1 0 0 1 1 1 7 7.029456
## 6 1 1 1 1 0 1 0 1 7 7.166172
## 6 1 1 1 0 1 0 1 1 7 7.246207
## 6 1 1 1 1 1 0 0 1 7 7.626958
## 6 1 1 1 1 0 0 1 1 7 8.314578
## 6 0 1 1 1 0 1 1 1 7 12.820254
## 6 0 1 1 1 1 1 0 1 7 12.825405
## 6 0 1 1 1 1 0 1 1 7 13.359493
## 6 0 1 1 0 1 1 1 1 7 19.340669
## 7 1 1 1 0 1 1 1 1 8 7.029455
## 7 1 1 1 1 1 1 0 1 8 7.735230

```

```
## 7 1 1 1 1 0 1 1 1 8 8.679263
## 7 1 1 1 1 1 0 1 1 8 8.921400
## 7 0 1 1 1 1 1 1 1 8 14.263216
## 7 1 1 1 1 1 1 1 0 8 26.311466
## 7 1 0 1 1 1 1 1 1 8 54.733440
## 7 1 1 0 1 1 1 1 1 8 75.293181
## 8 1 1 1 1 1 1 1 1 9 9.000000
```

Note that a model with x_1 , x_2 , x_3 , x_5 , and x_8 has a C_p that is near and above p .

3.10.2 AIC_p

The AIC criterion may be computed using `AIC()` with one or more fitted models. Compare models fitted to the same response and observations; smaller AIC is preferred. For example:

```
fit1 <- lm( lny ~ x1 + x2 + x3 + x5 + x8, data= surgunit.data )
fit2 <- lm( lny ~ x1 + x2 + x3 + x8, data= surgunit.data )
fit3 <- lm( lny ~ x1 + x2 + x4 + x6 + x8, data= surgunit.data )
AIC( fit1, fit2, fit3 )
```

```
##      df      AIC
## fit1  7 -8.020058
## fit2  6 -8.105992
## fit3  7 38.449552
```

3.10.3 BIC_p or SBC_p

Use `BIC()` to compute the BIC criterion. Equivalently, use `AIC()` with `k = log(nobs(fit1))` when all models use the same observations:

```
dim(surgunit.data)
```

```
## [1] 54 13
```

```
BIC(fit1, fit2, fit3)
```

```
##      df      BIC
## fit1  7  5.902830
## fit2  6  3.827912
## fit3  7 52.372440
```

In `AIC()`, the default penalty is $k = 2$; `BIC` uses $k = \log(n)$.

3.10.4 $AICc_p$

For these Gaussian linear models, compute Akaike's corrected information criterion ($AICc$) by adding $2K(K+1)/(n-K-1)$ to AIC . Here K counts estimated parameters, including the error variance; `AIC()` reports this count in its `df` column. The correction requires $n > K + 1$:

```
# get AIC of a model and store result
storeAIC <- AIC( fit1, fit2, fit3 )

AICc <- storeAIC$AIC + ( 2*(storeAIC$df)^2 + 2*storeAIC$df ) / ( nobs(fit1)- storeAIC$df -1)
AICc

## [1] -5.585275 -6.318758 40.884335

storeBIC <- BIC(fit1, fit2, fit3)

data.frame(model = rownames(storeAIC), AIC = storeAIC$AIC,
           AICc = AICc, BIC = storeBIC$BIC)

##  model      AIC      AICc      BIC
## 1  fit1 -8.020058 -5.585275  5.902830
## 2  fit2 -8.105992 -6.318758  3.827912
## 3  fit3 38.449552 40.884335 52.372440
```

3.10.5 Stepwise regression

The function `step()` in R performs forward and backward stepwise regression. The `step()` function uses AIC (by default) for its model selection criterion. The arguments of the step function are the initial model (an `lm` object) to start the procedure, the scope (the covariates under consideration), and the direction of the search ("backward", "forward", or "both")

Forward selection starts with a model with no explanatory variables and decides which covariates to add at each step by getting the explanatory variables that

gives the lowest AIC, if in the next step a lower AIC is not obtained, then it stops searching. Therefore, the model to specify in `step()` must be a model with just the intercept and the scope must include all the predictor variables to include in the search. We demonstrate with the Surgical Unit data but we limit ourselves to using the first four predictor variables for the purpose of illustration:

```
fit0 <- lm(lny ~ 1, data= surgunit.data) # model with just the intercept
# using AIC criterion
fit.fstepAIC <- step(fit0, scope=~ x1+x2+x3+x4, direction="forward" )
```

```
## Start:  AIC=-75.7
## lny ~ 1
##
##           Df Sum of Sq    RSS      AIC
## + x3      1     5.4762   7.3316 -103.827
## + x4      1     5.3990   7.4087 -103.262
## + x2      1     2.8285   9.9792  -87.178
## + x1      1     0.7763  12.0315  -77.079
## <none>                12.8077  -75.703
##
## Step:  AIC=-103.83
## lny ~ x3
##
##           Df Sum of Sq    RSS      AIC
## + x2      1     3.0191   4.3125 -130.48
## + x4      1     2.2019   5.1297 -121.11
## + x1      1     1.5506   5.7810 -114.66
## <none>                7.3316 -103.83
##
## Step:  AIC=-130.48
## lny ~ x3 + x2
##
##           Df Sum of Sq    RSS      AIC
## + x1      1     1.20395  3.1085 -146.16
## + x4      1     0.69836  3.6141 -138.02
## <none>                4.3125 -130.48
##
## Step:  AIC=-146.16
## lny ~ x3 + x2 + x1
##
##           Df Sum of Sq    RSS      AIC
## <none>                3.1085 -146.16
## + x4      1     0.024578  3.0840 -144.59
```

```
# model suggested by forward selection w/AIC
summary(fit.fstepAIC)
```

```
##
## Call:
## lm(formula = lny ~ x3 + x2 + x1, data = surgunit.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.46992 -0.17954 -0.03071  0.17883  0.59253
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.766176   0.226758  16.609 < 2e-16 ***
## x3           0.016452   0.001630   10.094 1.17e-13 ***
## x2           0.013340   0.002035    6.556 2.97e-08 ***
## x1           0.095458   0.021692    4.401 5.67e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2493 on 50 degrees of freedom
## Multiple R-squared:  0.7573, Adjusted R-squared:  0.7427
## F-statistic:    52 on 3 and 50 DF,  p-value: 2.124e-15
```

Now obtain results using the forward stepwise regression results using the BIC criterion:

```
# using BIC criterion --> note the argument k= log(n)
fit.fstepBIC <- step(fit0, scope=~ x1+x2+x3+x4, direction="forward" , k = log(nobs(fit0)) )
```

```
## Start:  AIC=-73.71
## lny ~ 1
##
##      Df Sum of Sq    RSS    AIC
## + x3   1    5.4762  7.3316 -99.849
## + x4   1    5.3990  7.4087 -99.284
## + x2   1    2.8285  9.9792 -83.200
## <none>          12.8077 -73.714
## + x1   1    0.7763 12.0315 -73.101
##
## Step:  AIC=-99.85
## lny ~ x3
##
##      Df Sum of Sq    RSS    AIC
```

```
## + x2      1      3.0191 4.3125 -124.516
## + x4      1      2.2019 5.1297 -115.146
## + x1      1      1.5506 5.7810 -108.691
## <none>                7.3316  -99.849
##
```

```
## Step: AIC=-124.52
```

```
## lny ~ x3 + x2
```

```
##
```

```
##           Df Sum of Sq    RSS    AIC
## + x1      1    1.20395 3.1085 -138.21
## + x4      1    0.69836 3.6141 -130.07
## <none>                4.3125 -124.52
##
```

```
## Step: AIC=-138.2
```

```
## lny ~ x3 + x2 + x1
```

```
##
```

```
##           Df Sum of Sq    RSS    AIC
## <none>                3.1085 -138.21
## + x4      1    0.024578 3.0840 -134.65
```

```
# model suggested by forward selection with BIC
summary(fit.fstepBIC)
```

```
##
```

```
## Call:
```

```
## lm(formula = lny ~ x3 + x2 + x1, data = surgunit.data)
```

```
##
```

```
## Residuals:
```

```
##      Min       1Q   Median       3Q      Max
## -0.46992 -0.17954 -0.03071  0.17883  0.59253
```

```
##
```

```
## Coefficients:
```

```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.766176   0.226758  16.609 < 2e-16 ***
## x3           0.016452   0.001630  10.094 1.17e-13 ***
## x2           0.013340   0.002035   6.556 2.97e-08 ***
## x1           0.095458   0.021692   4.401 5.67e-05 ***
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
```

```
## Residual standard error: 0.2493 on 50 degrees of freedom
```

```
## Multiple R-squared:  0.7573, Adjusted R-squared:  0.7427
```

```
## F-statistic:    52 on 3 and 50 DF,  p-value: 2.124e-15
```

Backward selection starts with a full model with all predictors and it decides which predictor to delete at each step by seeing which predictor deletion leads

to the lowest AIC, once a lower AIC is not obtained in the next step it stops. Therefore the model to specify in `step()` must be a model with all the predictor variables and the scope must include all the predictor variables to include in the search:

```
fitfull <- lm(lny ~ x1 + x2 + x3 + x4, data= surgunit.data) # full model with four predictors
# using AIC criterion
fit.BstepAIC <- step(fitfull, scope=~ x1+x2+x3+x4, direction="backward" )
```

```
## Start:  AIC=-144.59
## lny ~ x1 + x2 + x3 + x4
##
##           Df Sum of Sq    RSS    AIC
## - x4       1    0.0246 3.1085 -146.16
## <none>                        3.0840 -144.59
## - x1       1    0.5302 3.6141 -138.02
## - x2       1    1.8839 4.9678 -120.84
## - x3       1    3.4862 6.5702 -105.75
##
## Step:  AIC=-146.16
## lny ~ x1 + x2 + x3
##
##           Df Sum of Sq    RSS    AIC
## <none>                        3.1085 -146.161
## - x1       1    1.2040 4.3125 -130.483
## - x2       1    2.6724 5.7810 -114.658
## - x3       1    6.3341 9.4427  -88.162
```

```
# model suggested by backward selection with AIC
summary(fit.BstepAIC)
```

```
##
## Call:
## lm(formula = lny ~ x1 + x2 + x3, data = surgunit.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.46992 -0.17954 -0.03071  0.17883  0.59253
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.766176   0.226758  16.609  < 2e-16 ***
## x1           0.095458   0.021692   4.401 5.67e-05 ***
```

```
## x2          0.013340    0.002035    6.556 2.97e-08 ***
## x3          0.016452    0.001630   10.094 1.17e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2493 on 50 degrees of freedom
## Multiple R-squared:  0.7573, Adjusted R-squared:  0.7427
## F-statistic:    52 on 3 and 50 DF,  p-value: 2.124e-15
```

Now obtain results using the backward stepwise regression results using the BIC criterion:

```
# using BIC criterion --> note the argument k= log(n)
fit.BstepBIC <- step(fitfull, scope=~ x1+x2+x3+x4, direction="backward" , k = log(nob
```

```
## Start:  AIC=-134.64
## lny ~ x1 + x2 + x3 + x4
##
##           Df Sum of Sq    RSS      AIC
## - x4      1     0.0246  3.1085 -138.205
## <none>                    3.0840 -134.645
## - x1      1     0.5302  3.6141 -130.067
## - x2      1     1.8839  4.9678 -112.888
## - x3      1     3.4862  6.5702  -97.792
##
## Step:  AIC=-138.2
## lny ~ x1 + x2 + x3
##
##           Df Sum of Sq    RSS      AIC
## <none>                    3.1085 -138.205
## - x1      1     1.2040  4.3125 -124.516
## - x2      1     2.6724  5.7810 -108.691
## - x3      1     6.3341  9.4427  -82.195
```

```
# model suggested by backward selection with BIC
summary(fit.BstepBIC)
```

```
##
## Call:
## lm(formula = lny ~ x1 + x2 + x3, data = surgunit.data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.46992 -0.17954 -0.03071  0.17883  0.59253
```

```
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 3.766176   0.226758  16.609 < 2e-16 ***
## x1          0.095458   0.021692   4.401 5.67e-05 ***
## x2          0.013340   0.002035   6.556 2.97e-08 ***
## x3          0.016452   0.001630  10.094 1.17e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2493 on 50 degrees of freedom
## Multiple R-squared:  0.7573, Adjusted R-squared:  0.7427
## F-statistic:    52 on 3 and 50 DF,  p-value: 2.124e-15
```

3.10.6 CV/PRESS and GCV

Leave-one-out cross-validation assesses prediction error. PRESS is the sum of squared leave-one-out prediction errors, while the CV score used here is PRESS divided by the number of observations. We use the computational shortcut $e_i/(1-h_{ii})$ for each leave-one-out residual. To illustrate, the CV scores of `fit1`, `fit2`, and `fit3` are computed.

```
n <- nobs(fit1)

### CV score fit1
cvtop <- residuals( fit1 )

# diagonal elements of the hat matrix: h_ii's
cvbottom <- 1- hatvalues( fit1 )

# CV score
CVfit1 <- 1/n*sum( (cvtop/cvbottom)^2 )
###
###

### CV score fit2
cvtop <- residuals( fit2 )

# hatvalues # <--- provides the diagonal elements of H
cvbottom <- 1- hatvalues( fit2 )

# CV score
CVfit2 <- 1/n*sum( (cvtop/cvbottom)^2 )
###
```

```
###

### CV score fit3
cvtop <- residuals( fit3 )

# diagonal elements of the hat matrix: h_ii's
# hatvalues # <--- provides the diagonal elements of H
cvbottom <- 1- hatvalues( fit3 )

# CV score
CVfit3 <- 1/n*sum( (cvtop/cvbottom)^2 )
###
###

CVfit1
```

```
## [1] 0.05072096
```

```
CVfit2
```

```
## [1] 0.05069947
```

```
CVfit3
```

```
## [1] 0.1158974
```

Next, compute the GCV scores:

```
n <- nobs(fit1)

### GCV score fit1
### sum of squared residuals
SSE <- sum( residuals( fit1 )^2 )

gcvbottom <- ( 1- mean( hatvalues( fit1 ) ) )^2

# GCV score
gcvfit1 <- 1/n*( SSE/gcvbottom )
###
###
```

```

### GCV score fit2
### sum of squared residuals
SSE <- sum( residuals( fit2 )^2 )

gcvbottom <- ( 1- mean( hatvalues( fit2 ) ) )^2

# GCV score
gcvfit2 <- 1/n*( SSE/gcvbottom )
###
###

### GCV score fit3
### sum of squared residuals
SSE <- sum( residuals( fit3 )^2 )

gcvbottom <- ( 1- mean( hatvalues( fit3 ) ) )^2

# GCV score
gcvfit3 <- 1/n*( SSE/gcvbottom )
###
###

gcvfit1

```

```
## [1] 0.04928725
```

```
gcvfit2
```

```
## [1] 0.04900256
```

```
gcvfit3
```

```
## [1] 0.1165376
```

3.11 More diagnostics

3.11.1 Identifying outlying y and x values

The body fat example is used to illustrate studentized residuals, deleted residuals, and studentized deleted residuals. To start, import the data:

```
### Import data:
url <- "https://www.csub.edu/~emontoya2/datasets/textbookdata/Kutner/Chapter%20%207%201"

### This is not a .csv file and the file does not
### include variable names, so some changes
### have to be made to read.csv().
bf.data <- read.csv( url , header = FALSE , col.names = c( "x1", "x2", "x3", "y" ) , as.is = TRUE )

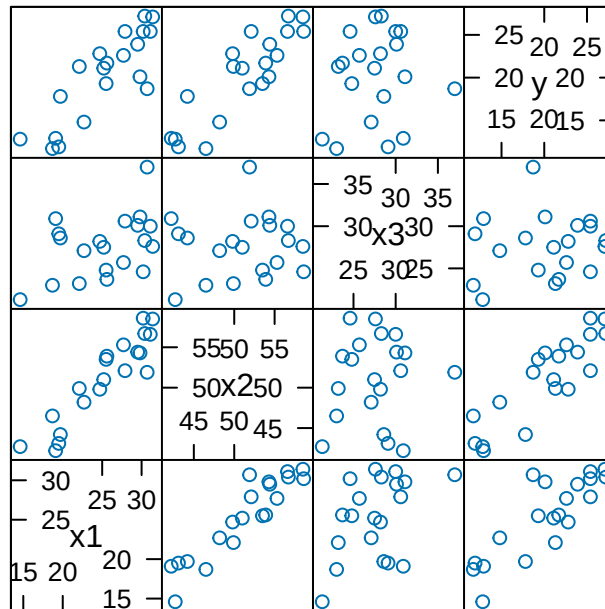
pillar::glimpse( bf.data )
```

```
## Rows: 20
## Columns: 4
## $ x1 <dbl> 19.5, 24.7, 30.7, 29.8, 19.1, 25.6, 31.4, 27.9, 22.1, 25.5, 31.1, 3~
## $ x2 <dbl> 43.1, 49.8, 51.9, 54.3, 42.2, 53.9, 58.5, 52.1, 49.9, 53.5, 56.6, 5~
## $ x3 <dbl> 29.1, 28.2, 37.0, 31.1, 30.9, 23.7, 27.6, 30.6, 23.2, 24.8, 30.0, 2~
## $ y <dbl> 11.9, 22.8, 18.7, 20.1, 12.9, 21.7, 27.1, 25.4, 21.3, 19.3, 25.4, 2~
```

```
#> Rows: 20
#> Columns: 4
#> $ x1 <dbl> 19.5, 24.7, 30.7, 29.8, 19.1, 25.6, 31.4, 27.9, 22.1, 25.5, 31.1, 3~
#> $ x2 <dbl> 43.1, 49.8, 51.9, 54.3, 42.2, 53.9, 58.5, 52.1, 49.9, 53.5, 56.6, 5~
#> $ x3 <dbl> 29.1, 28.2, 37.0, 31.1, 30.9, 23.7, 27.6, 30.6, 23.2, 24.8, 30.0, 2~
#> $ y <dbl> 11.9, 22.8, 18.7, 20.1, 12.9, 21.7, 27.1, 25.4, 21.3, 19.3, 25.4, 2~
```

Examine the scatterplot matrix for potential outlying values:

```
splom(bf.data)
```



Scatter Plot Matrix

To obtain standardized (internally studentized) residuals, use `rstandard()` with an *lm* object as its first argument:

```
fit1 <- lm( y ~ x1 + x2 + x3, data= bf.data)

# standardized (internally studentized) residuals
rstandard( fit1 )
```

```
##          1          2          3          4          5          6
## -1.46802633  1.13326956 -1.23262045 -1.29571232  0.57630252 -0.23525791
##          7          8          9         10         11         12
##  0.62249950  1.38022830  0.76529502 -0.57761774  0.34965240  0.94324119
##         13         14         15         16         17         18
## -1.50477923  1.84715613  0.49352568  0.07392664 -0.16107977 -0.63614383
##         19         20
## -1.61308352  0.25538013
```

The function `rstudent()` obtains externally studentized (studentized deleted) residuals, using the error-variance estimate from a fit with that case omitted:

```
# studentized deleted residuals
rstudent( fit1 )
```

```
##          1          2          3          4          5          6
## -1.52803951  1.14416429 -1.25452990 -1.32606735  0.56388572 -0.22818249
##          7          8          9         10         11         12
##  0.61016668  1.42385079  0.75493974 -0.56519997  0.33985038  0.93979234
##         13         14         15         16         17         18
## -1.57251203  2.01637183  0.48153342  0.07159138 -0.15609143 -0.62388391
##         19         20
## -1.70680191  0.24777625
```

The function `hatvalues()` returns leverage values (h_{ii}), the diagonal entries of the hat matrix. Leverage measures how unusual the predictor values are; it is not itself a measure of influence. Here, a leverage above $3p/n$ is a screening rule, with p counting fitted coefficients including the intercept:

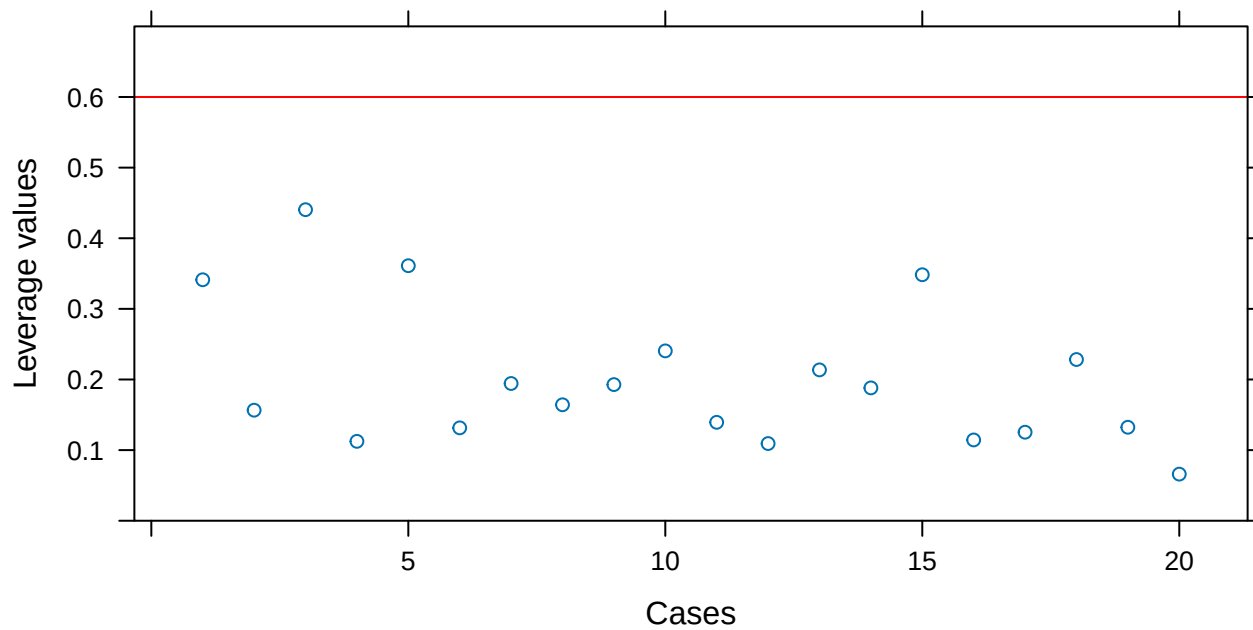
```
# leverage values
hatvalues( fit1 )
```

```
##          1          2          3          4          5          6          7
## 0.34120920 0.15653638 0.44042770 0.11242972 0.36109984 0.13151364 0.19433721
##          8          9         10         11         12         13         14
## 0.16418081 0.19278940 0.24051819 0.13935816 0.10929380 0.21357666 0.18808377
##         15         16         17         18         19         20
## 0.34830629 0.11439069 0.12532943 0.22828343 0.13235798 0.06597771
```

```
n.cases <- nobs(fit1)
p.coef <- fit1$rank # includes the intercept
leverage.cutoff <- 3 * p.coef / n.cases
leverage.cutoff
```

```
## [1] 0.6
```

```
xyplot( hatvalues( fit1 ) ~ seq_len(n.cases) ,
        panel = function(x, y) {
          panel.xyplot(x, y)
          panel.abline(a=leverage.cutoff, b=0, col="red")
        }, xlab = "Cases",
        ylab = "Leverage values", ylim=c(0, .7))
```



3.11.2 Identifying influential cases

DFFITS measures the standardized change in a case's fitted value when that case is omitted. Obtain it with `dffits()`. Inspect its absolute value, since influential cases may have positive or negative DFFITS. One screening rule is $2\sqrt{p/(n-p)}$, where p includes the intercept (Penn State's regression notes); thresholds guide investigation rather than establish influence:

```
dffits( fit1 )
```

```
##          1          2          3          4          5          6
## -1.09969231  0.49290457 -1.11298735 -0.47195954  0.42392423 -0.08879453
##          7          8          9         10         11         12
##  0.29967452  0.63105834  0.36894399 -0.31806611  0.13675485  0.32920196
##         13         14         15         16         17         18
## -0.81948815  0.97048913  0.35203458  0.02572972 -0.05908583 -0.33932204
##         19         20
## -0.66663467  0.06585362
```

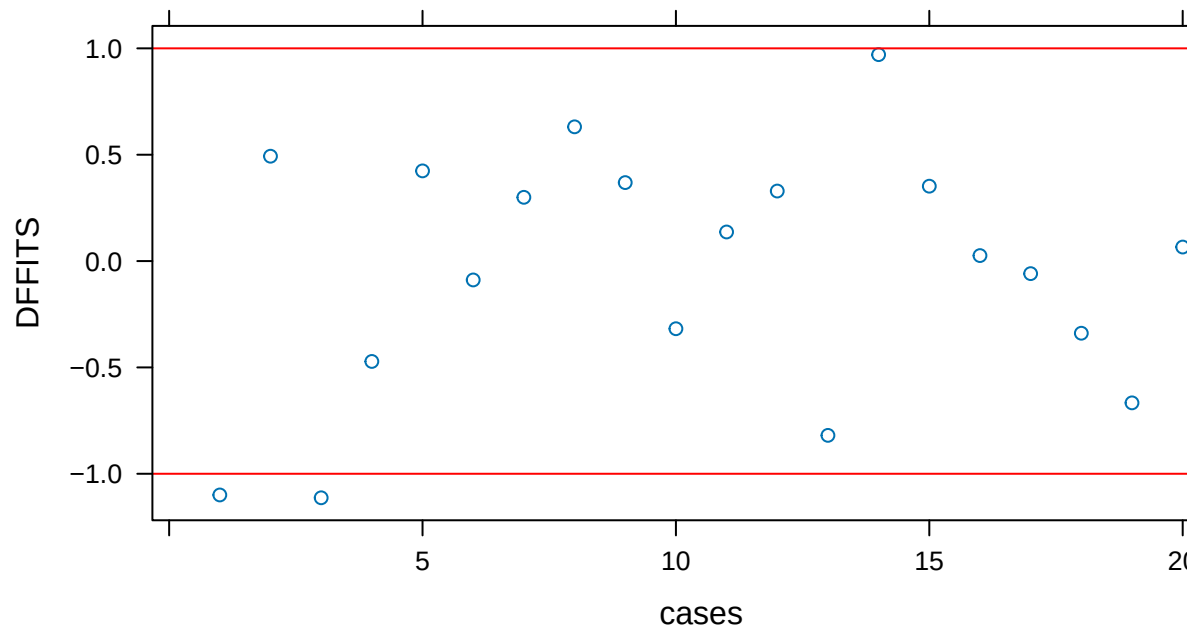
```

dffits.values <- dffits(fit1)
dffits.cutoff <- 2 * sqrt(p.coef / (n.cases - p.coef))
dffits.cutoff

## [1] 1

# plot DFFITS vs cases
xyplot( dffits.values ~ seq_len(n.cases),
  panel = function(x, y) {
    panel.xyplot(x, y)
    panel.abline(h=c(-dffits.cutoff, dffits.cutoff), col="red")
  },
  xlab = "cases",
  ylab = "DFFITS",
  ylim=extendrange(c(dffits.values, -dffits.cutoff, dffits.cutoff)) )

```



Cook's distance measure is computed using `cooks.distance()` and it requires an *lm* object:

```
cook.values <- cooks.distance(fit1)
cook.values
```

```
##           1           2           3           4           5           6
## 0.2790492817 0.0595875286 0.2989628090 0.0531661934 0.0469283564 0.0020952519
##           7           8           9          10          11          12
## 0.0233679530 0.0935517519 0.0349698759 0.0264150596 0.0049490632 0.0272927035
##          13          14          15          16          17          18
## 0.1537386238 0.1976001749 0.0325444771 0.0001764779 0.0009294597 0.0299272765
##          19          20
## 0.0992346339 0.0011517393
```

```
# The reference lines at 0.5 and 1 are screening guides.
# Investigate large values in context; do not automatically delete cases.
```

```
xyplot( cook.values ~ seq_len(n.cases),
        panel = function(x, y) {
          panel.xyplot(x, y)
          panel.abline(a=c(.5), b=c(0), col="red")
          panel.abline(a=c(1), b=c(0), col="blue")
        },
        xlab = "cases",
        ylab = "Cook's distance", ylim=c(0, max(1.2, cook.values)) )
```

